中山大学硕士学位论文

基于 iOS 可视化操作的六足机器人设计与实现

Design and implementation of Hexapod robot base on iOS platform with Visualization control

专　　业：　计算机技术

学位申请人：

指导教师：

论文答辩委员会主席：_____

成员：_____

_____

_____

二 0 一六 年 十一 月

原创性及使用授权声明

论文原创性声明内容

本人郑重声明：所呈交的学位论文，是本人在导师的指导下，独立进行研究工作所取得的成果。除文中已经注明引用的内容外，本论文不包含任何其他个人或集体已经发表或撰写过的作品成果。对本文的研究作出重要贡献的个人和集体，均已在文中以明确方式标明。本人完全意识到本声明的法律结果由本人承担。

学位论文作者签名：

日期：2016 年 11 月　　日

学位论文使用授权声明

本人完全了解中山大学有关保留、使用学位论文的规定，即：学校有权保留学位论文并向国家主管部门或其指定机构送交论文的电子版和纸质版；有权将学位论文用于非赢利目的的少量复制并允许论文进入学校图书馆、院系资料室被查阅；有权将学位论文的内容编入有关数据库进行检索；可以采用复印、缩印或其他方法保存学位论文；可以为存在馆际合作关系的兄弟高校用户提供文献传递服务和交换服务。

保密论文保密期满后，适用本声明。

学位论文作者签名：　　导师签名：

日期：2016 年 11 月　日　　日期：2016 年 11 月　日

基于 iOS 可视化操作的六足机器人设计与实现

专业： 计算机技术

硕士生： 余冠源

指导教师：张治国副教授

中文摘要

机器人研究一直是业内一个热门的方向，但是机器人的入门门槛相对较高，研究人员需要有扎实的机械、电子和编程知识。针对这个问题，微软推出的机器人开发平台 Microsoft Robotics Studio，还有乐高公司的 Robolab 也是针对入门开发者开发的一套可视化开发环境，而斯坦福大学的人工智能实验室则开源了一套名为 Robot Operating System (ROS)的机器人开发框架。这些工具都很好的弥补了机器人可视化开发和操作的市场空缺。

虽然市场上已经有类似的产品出现，但是像微软和乐高公司的这些开发工具，操作仍然相对复杂，同时乐高产品的硬件平台费用相对较高，而微软的开发环境则依赖于 windows 操作系统。而对于儿童或入门级别的用户而言，仍然是门槛较高。斯坦福大学的 ROS 框架则基于 Linux Ubuntu 操作系统，对于没有 Linux 开发经验的用户来说，也是门槛较高。

本文旨在构建一个基于 iOS 系统的 app 应用，下位机基于 STM32，构造一个可视化的，操作更简易的，适合儿童或零基础的用户使用的一款六足机器人。

本课题利用了当下流行的智能手机 iPhone，通过蓝牙 4.0 通信模块，实现 app 与六足机器人下位机的交互，同时定义了一套简单的操作指令，可以让用户在 app 中方便的操作，定义动作，并下载至控制板上。同时，六足机器人下位机实现了一个简单的文件系统，可以让用户方便的保存动作数据，同时可以支持 bootloader，无线升级固件，免去繁琐的线缆连接。

本课题通过对 app 实现简易的可视化操作的完整方案的探讨和研究，构建出一个适合入门用户或儿童的一套产品。用户无需学习任何编程知识，即可操作机器人，可以通过简单的几个命令的组合，即可编排出复杂的动作，从而培养用户对机器人的兴趣，为进一步的学习和研究奠定基础。

虽然本课题的实现了一个基本的六足机器人可视化操作 app，但是支持的命令还非常有限，对于打算对这个课题进行深入研究的研究者来说，可以根据本课题的思想，继续深入探讨，并加入更多的，更复杂的操作指令，从而使得用户可以编排出更多更有想象力的作品。

关键字：六足机器人，蓝牙控制，iOS 可视化操作，六足机器人动作编辑

Design and implementation of Hexapod robot base on iOS platform with Visualization control

Major: Computer Technology
Name: Yu Guanyuan
Supervisor: Zhang Zhiguo

ABSTRACT

Robot research is always a hot topic in industry. However, to do research on robotics, you need an experience technical background in computer programming, electric knowledge, and mechanical experience as well. To solve this problem, Microsoft published a toolkit named Microsoft Robotics Studio to help beginners to build robot application with visualization. Lego also announce a platform named Robolab, which also help beginners to build fantastic robot applications. For the experienced users, Stanford University has opened source a robot framework named Robot Operating System, also known as ROS. These excellent tools are filling what the market's needs on robot visualization control.

However, most of these tools are still too complicated to handle for the children or the user without any programming background. Robolab is relied on Lego's hardware, which is expensive to most of the users. Microsoft Robotics Studio runs on Windows system. And ROS is built upon Linux Ubuntu system, so if user without any Linux programming background, ROS is hard to mastered.

This article aims to build an app on top of iOS system, and the robot control board is built with STM32 MCU. This app is tried to build a visualization robot control platform, and being an app with more easy to use functionalities, comforted to the children and the beginners. This system is built with iOS platform, and making used of Bluetooth 4.0 modules, to implement control between app and the control board. And this system defined a simple instruction set, letting the users to define motions in the app. And the motion definition could also be downloaded to the control board via BLE, so the users may not need the wires any more.

We have discussed the solution for robot visualization control system here, and build a system based on iOS and hexapod robot. Users with or without any programming background could easily create and manipulate robot motions with this app. This is a good start for the users interested in robotics, and this system may help them to master the basic knowledge about robotics, and with these knowledge, they may go further in this field.

Although we have built a system with visualization control on hexapod robot, the support commands for this app is too limited. For the researchers on this topic, they could follow my methodology, and extends the command set, and develop more complicated instructions, to help users build more fantastic works.

Key words: Hexapod, Bluetooth LE Control, iOS Visualization Control, Hexapod Motion Editor

目录

中文摘要.....I
英文摘要.....III
目录.....V
第 1 章 绪论.....1
1.1 研究背景与意义.....1
1.2 可视化控制软件的研究和应用现状.....1
1.3 六足机器人的研究和应用现状.....3
1.4 本文研究内容.....4
第 2 章 基于 iOS 可视化操作的六足机器人的需求分析.....5
2.1 六足机器人的硬件平台需求.....5
2.2 六足机器人的关键技术点.....6
2.3 六足机器人的通信需求.....7
2.4 六足机器人的嵌入式系统需求.....7
2.5 六足机器人控制 App 的需求.....8
第 3 章 基于 iOS 可视化操作的六足机器人的架构设计.....9
3.1 硬件平台架构.....9
3.2 软件平台架构.....10
3.3 Objective-C 的介绍.....13
3.4 iOS 客户端使用的 framework.....14
3.5 iOS app 架构.....15
第 4 章 基于 iOS 可视化操作的六足机器人的下位机系统实现.....16
4.1 HAL 硬件驱动层.....16
4.1.1 舵机驱动.....16
4.1.2 蓝牙驱动.....17
4.1.3 ADC 采样驱动.....19
4.1.4 LED/蜂鸣器驱动.....20
4.1.5 Flash 读写驱动.....20
4.1.6 定时器驱动.....21
4.2 应用服务层.....22
4.2.1 蓝牙帧处理 RFHandler.....22
4.2.2 动作处理单元 RBMotion.....27
4.2.3 文件系统 RBFS.....30
4.2.4 系统服务 System.....31
4.3 系统线程.....32
4.4 本章小结.....34
第 5 章 基于 iOS 可视化操作的六足机器人的 app 实现.....35
5.1 RobotScratch 架构总揽.....35
5.2 iOS 蓝牙实现.....36
5.2.1 技术背景介绍.....36

5.2.2 蓝牙工具类 BLEManager 的设计与实现 36
5.3 六足机器人 UI 界面的实现 40
5.3.1 Console 控制台的实现 40
5.3.2 Gamepad 模拟遥控手柄界面的实现 41
5.3.3 MotionEditor 动作编辑器的实现 41
5.3.4 固件 Firmware 升级的实现 42
5.4 本章小结 44
第 6 章 基于 iOS 的可视化操作的六足机器人的集成测试 45
6.1 蓝牙 4.0 通信集成测试 45
6.1.1 测试方法 45
6.1.2 测试结果 45
6.2 六足机器人固件在线升级测试 47
6.2.1 测试方法 47
6.2.2 测试结果 48
6.3 iOS 客户端控制台舵机控制测试 50
6.3.1 测试方法 50
6.3.2 测试结果 50
6.4 iOS 客户端 MotionEditor 动作编辑测试 51
6.4.1 测试方法 51
6.4.2 测试结果 51
6.5 iOS 客户端远程参数配置测试 52
6.5.1 测试方法 52
6.5.2 测试结果 52
6.6 六足机器人动作测试 52
6.6.1 测试方法 52
6.6.2 测试结果 54
6.7 本章小结 56
第 7 章 结论与展望 57
致谢 59

第 1 章 绪论

1.1 研究背景与意义

六足机器人是一种受昆虫生物启发的仿生机器人，例如卡梅隆大学的 RiSE，NASA 研发的 ATHLETE 6 自由度仿生机器人[1]。六足机器人相对于二足机器人有更好的平衡性，对于入门级别的机器人来说，开发的难度也会相对较低。

为了让儿童或者没有编程基础的用户接触并使用机器人，本课题通过构造一个基于 iOS 的可视化操作的 app，让用户可以更加简单快捷的编排机器人的动作，通过简单的几个命令组合出令人意想不到的复杂动作。同时本课题采用蓝牙控制，免去了传统机器人控制需要的线缆连接，从而有更好的机动性。另外本课题的 app 建立与 iOS 平台之上，随着 iPhone，iPad 智能手机，平板的普及，可以让缺乏电脑知识的儿童也可以更好的操作六足机器人。

本课题提出了以下几个问题，并通过设计和实现六足机器人，解决这些问题：

（1）六足机器人需要实现上位机和下位机，实现控制功能。

（2）上位机与下位机需要实现无线通信。

（3）下位机的六足机器人必须有很好的实时性和可靠性。

（4）下位机必须有一个容错性很好的固件升级功能。

接下来将从国内外对六足机器人及其控制软件研究作出分析和讨论，最后引出本课题的设计和实现方案。

1.2 可视化控制软件的研究和应用现状

微软在 08 年发布了一款名为 Microsoft Robotics Developer Studio （简称 MRDS）的基于 windows 的机器人开发环境和模拟器[2]。这款软件面向学校，兴趣爱好者和一些商业用户。MRDS 基于.net 的一个并行库 CCR 而开发的，可以管理异步并行任务。MRDS 包含一个可视化操作界面，并运用 VPL 作为开发语言[3]。图 1-1 为 MRDS 的模拟器：

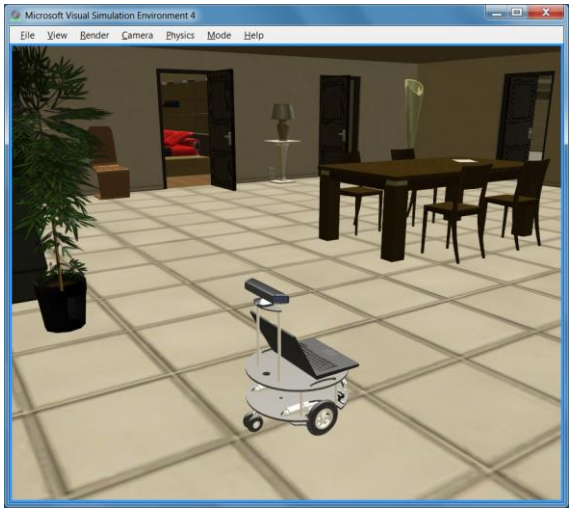

图 1-1MDRS 模拟器

微软的 MDRS 是一款面向于有编程基础的用户，对于儿童或零编程基础的用户来说，不容易掌握。

乐高 LEGO 公司也针对机器人爱好推出了一款名为 Robolab 的软件，旨在为乐高用户提供一个实现可视化编程的界面，基于 RCX 核心[2]。但是使用 Robolab 必须搭载乐高公司的机器人。通过 Robolab，用户可以通过鼠标拖拉控件，实现类似于程序的流程图。图 1-2 为 Robolab 界面：

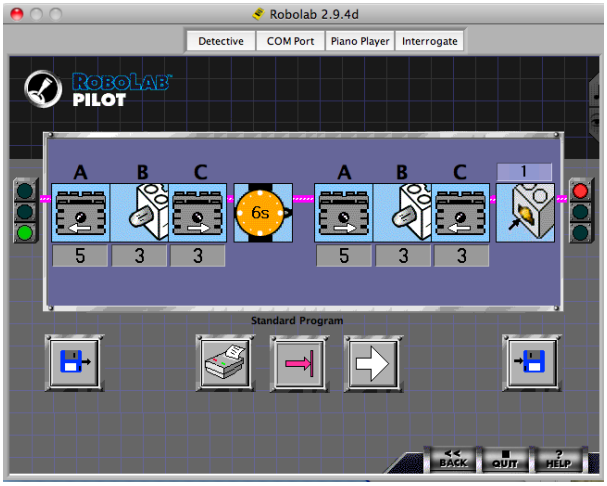

图 1-2 乐高公司 Robolab 界面截图

乐高的 Robolab 操作简单，图形化的控件管理，建立类似于程序流程图的可视化程序。但是 Robolab 的硬件价格相对较高，而对于儿童或零基础的用户来说，操作还是相对繁琐。

斯坦福大学人工智能实验室在 2013 年开源了面向于机器人构建的框架，名为 Robot Operating System（ROS）。ROS 基于计算机集群，提供类似于操作系统的功能，ROS 提供了标准的硬件抽象层，底层硬件驱动，并实现了常用的功能，如进程间的消息传递，包管理，基于图形化的节点模拟等功能。ROS 是一个面向专业人士开发的一套机器人开发框架，其建立于 Linux Ubuntu 之上，所以入门门槛相对较高，缺乏编程经验的人，使用起来会非常困难。

1.3 六足机器人的研究和应用现状

美国波士顿动力公司的 RHex 是一个具有很高移动性能的六足机器人[4]，RHex 如图 1-3 所示。RHex 拥有一个海豹的躯体，可以让它在潮湿的地方作业，例如沼泽，泥泞的区域，人类无法容易行走的区域。RHex 可以通过无线实现长达 700 米的远程控制。而且 RHex 还附带了摄像头实现正面和侧面的实时画面传输。

图 1-3 波士顿动力 RHex

LAURON 是一个由德国 Forschungszentrum Informatik Karlsruhe (FZI)研发机构开发的六足步行机器人[5]。LAURON 第一代在 90 年代初面世，并于 1994 年的 CeBIT 展向公众展示。第一代 LAURON 是通过神经网络算法实现控制，最新的第五代已于 2013 年完成，机器人如图 1-4 所示。LAURON 的研发旨在研究在崎岖地形上的机器人平衡问题。LAURON 搭载了一个摄像头，可以方便操纵者观察机器人所处的地形，同时 LAURON 安装了可以搜集环境信息的传

感器，并可以通过设定目标实现自动线路规划。LAURON 主要是开发用于在人类进入困难的地带区域执行勘查任务。

图 1-4 第五代 LAURON 六足机器人

1.4 本文研究内容

本文阐述基于 iOS 平台构建可视化操作六足机器人，实现舵机的保护，实时控制板健康状态检查，图形化编程，简易的图形化操作，需要研究如下内容：

（1）六足机器人控制板硬件平台的搭建与开发。其中包括硬件控制板的设计与嵌入式系统的开发。

（2）蓝牙 4.0 通信协议的编写与实现，包括下位机舵机控制板的协议实现，与 app 平台的蓝牙通信协议实现。

（3）动作单元模块的编写，动作指令定义，动作帧协议的开发。包括下位机舵机控制板的实现和 App 可视化编程界面的开发。

（4）舵机控制板的实时健康检查，包括电流电压的采样。App 平台的健康数据展示逻辑。

（5）舵机控制板的文件系统实现。

（6）舵机控制板 Bootloader 机制的实现，和 IAP 固件升级机制。

第 2 章 基于 iOS 可视化操作的六足机器人的需求分析

2.1 六足机器人的硬件平台需求

主控选择 RobotBros 开发的 SPIDERX-NANO-T74 舵机控制开发板，如图 2-1 所示，开发板板载 22 路 PWM 舵机控制 I/O 口，并附带通用串口接口，用户可以自由选择串口蓝牙模块，实现与手机 app 的无线通信。同时 T74 使用开关电源替代传统的电桥方案，可以降低能耗，提高电源效率。

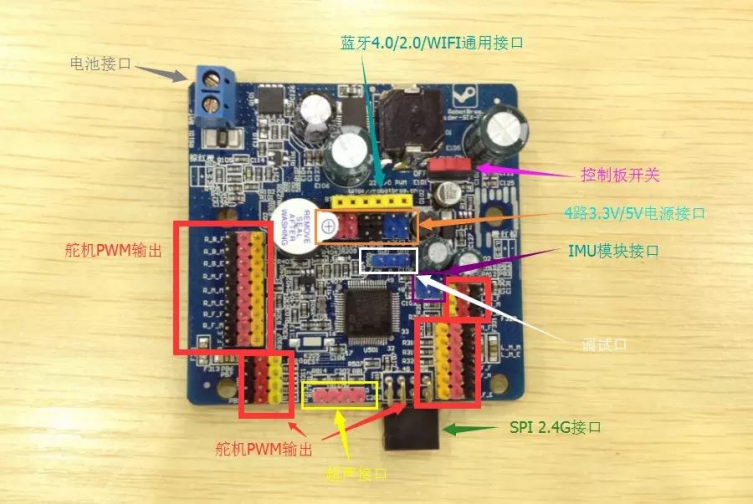

图 2-1 T74 开发板外设接口示意图

蓝牙模块选择的是基于 TI 公司的蓝牙 4.0 芯片的 CC2540 的串口通信模块，如图 2-2 所示。CC2540 集成了 8051 单片机，同时提供丰富的外设接口，如串口和 SPI 接口[6]，而且功耗控制也是业内领先，另外 CC2540 还有大量的开发资料和社区支持。

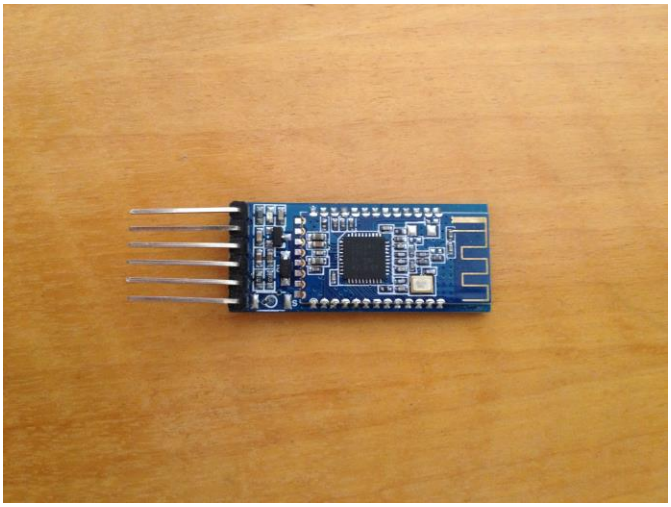

图 2-2 CC2540 蓝牙 4.0 模块

2.2 六足机器人的关键技术点

六足机器人系统需要实现多线程，舵机转动需要同步进行，同时需要执行电流电压采样实时保护舵机。当今流行的嵌入式系统有开源实时操作系统 FreeRTOS，国产实时嵌入式系统 RT-Thread，专门面向物联网的 Contiki，商业授权的有 uCOS-II, QNX，Windows-CE，VxWroks。在各大嵌入式操作系统中，开源的系统备受青睐，特别是 FreeRTOS，具有丰富的文档支持和开发者社区支持。同时 FreeRTOS 可以轻易的移植到 STM32 平台，具有操作系统基本的模块，例如邮箱，信号量，互斥锁等[7]。所以 FreeRTOS 非常适用于应用在六足机器人上。所以本项目选用的是 FreeRTOS。

App 控制六足机器人的其中一个关键技术点在于蓝牙通信的实现。蓝牙通信需要定义一套命令帧，下位机根据 App 的指令执行不同的操作。

App 实现可视化的动作编辑操作，也必须定义一套指令，实现 App 本地编辑，远程下载，远程执行动作。

最后一个关键技术点在于下位机的固件在线升级，通过 App 的蓝牙，实现远程固件升级，升级容错功能。

2.3 六足机器人的通信需求

为了实现无信通信，有多种方式可选。如 WIFI 无线通信，2.4G 无线通信和低功耗蓝牙通信。蓝牙 4.0 也称为低功耗蓝牙(BLE)，是一种现在普遍用于近距离通信的协议，例如个人健康产品，娱乐产品等[8]。而且目前大部分智能手机都已支持蓝牙 4.0 模块。例如苹果的 iPhone 从 4s 开始就已集成了蓝牙 4.0 模块。同时蓝牙 4.0 芯片的单块成本也在不断下降。以下将对上述三种通信方式进行比较：

WIFI 通信的优劣：

（1）数据传输速度快，传输距离可以达 100 米

（2）模块成本较高，下位机和上位机的软件实现比较复杂

（3）WIFI 传输功耗较大

2.4G 无线通信的优劣

（1）传输距离远，可以达 1000 米

（2）传输速度较慢

（3）上位机需要配备 2.4G 对应的通信模块，成本会相应增加

低功耗蓝牙通信的优劣：

（1）传输距离在 10 米左右

（2）功耗低，适合使用在通信数据量较小的场合

（3）市场上大部分智能手机已支持蓝牙 4.0，可以利用现有的模块

（4）软件实现相对简单

由于六足机器人需要在数据吞吐量与功耗上做出平衡，而六足机器人的数据传输量较少，综合以上的对比，在本项目中，将使用蓝牙 4.0 协议作为通信协议。

2.4 六足机器人的嵌入式系统需求

（1）实时电流电压监测：对于一般的六足机器人，舵机是最昂贵的配件之一。而一般的舵机，因为成本问题都没有防烧毁的电路，所以为了对舵机的保护，提高产品的可靠性与寿命，六足机器人嵌入式系统需实现了电流电压的峰

值保护，系统实时对电路的电压和电流采样，若超过特定阈值，则自动切段电源，防止舵机的烧毁。

（2）App 对实时健康数据的显示，包括电量信息，电流值。

（3）六足机器人嵌入式系统需及时相应 App 的蓝牙指令，并执行相应的动作。

（4）文件系统的实现：App 编排的动作，需要保存到六足机器人的 flash 模块中，实现掉电的数据保存。并允许用户实现文件系统的操作，包括查看，创建，更新，删除操作。

（5）Bootloader 功能：六足机器人人开发过程中，经常涉及到固件的更新，而在产品量产后，固件更新变得尤为重要。六足机器人嵌入式系统必须支持开机自检功能，并实现程序的引导操作。

2.5 六足机器人控制 App 的需求

为了让零编程基础的六足机器人入门爱好者更好的使用，降低操作六足机器人的入门门槛，App 需要以更友好的图形提供给使用者，让使用者更好的操作六足机器人。App 需实现以下四大功能：

（1）以图形化的方式，对每个舵机的角度实现控制，并支持实时角度的同步操作。

（2）以手柄操作模式，实现六足机器人四个方向的控制，包括向前，向后，想做向右，并可以根据模拟手柄实现速度的控制。

（3）动作编辑器。动作编辑器是以图形化的界面实现六足机器人动作的编排。

（4）App 在线对开发板固件的升级。普通的用户，因为缺乏编程的能力，若六足机器人需要更新固件，那么就变得非常麻烦，App 需要能够提供在线 IAP 固件升级机制，可以在六足机器人上电运行过程中实现升级。

第 3 章 基于 iOS 可视化操作的六足机器人的架构设计

3.1 硬件平台架构

硬件平台采用基于 RobotBros 生产的 XSPIDER 开发板，该开发板主控 MCU 为 STM32F103RC，256K Flash，48K RAM，4 个定时器，硬件平台构架如图 3-1 所示：

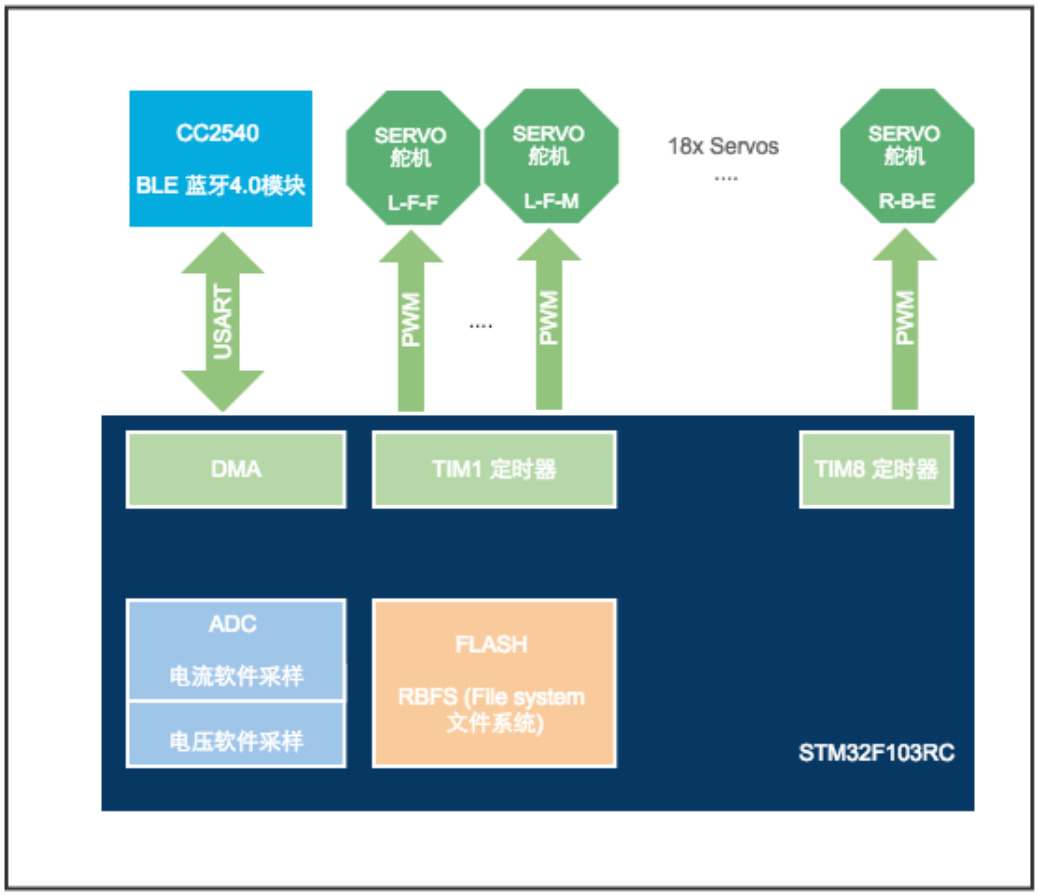

图 3-1 硬件平台构架图

六足机器人，每只腿配有 3 个舵机，总共 18 个舵机。舵机采用银燕 9 克舵机 ES08MAII。STM32F103RC 通用定时器支持 3 路 PWM 输出。

蓝牙模块使用 TI 芯片的 CC2540 为核心的蓝牙透传模块，可以实现模块的积木化，用户可以根据自己的喜好采购蓝牙模块，该模块同时支持 AT 指令配

置的透传通信模式。大大简化开发的难度。蓝牙模块通过串口 USART 实现与主控的通信。

STM32F103RC 具有 12 位 ADC 模数转换组件，可以实现电压/电流的采样[9]。

舵机命名规则如图 3-2 所示：

关节趾端 E，关节中间连接 M，关节前端舵机 F，左脚 L，右脚 R。

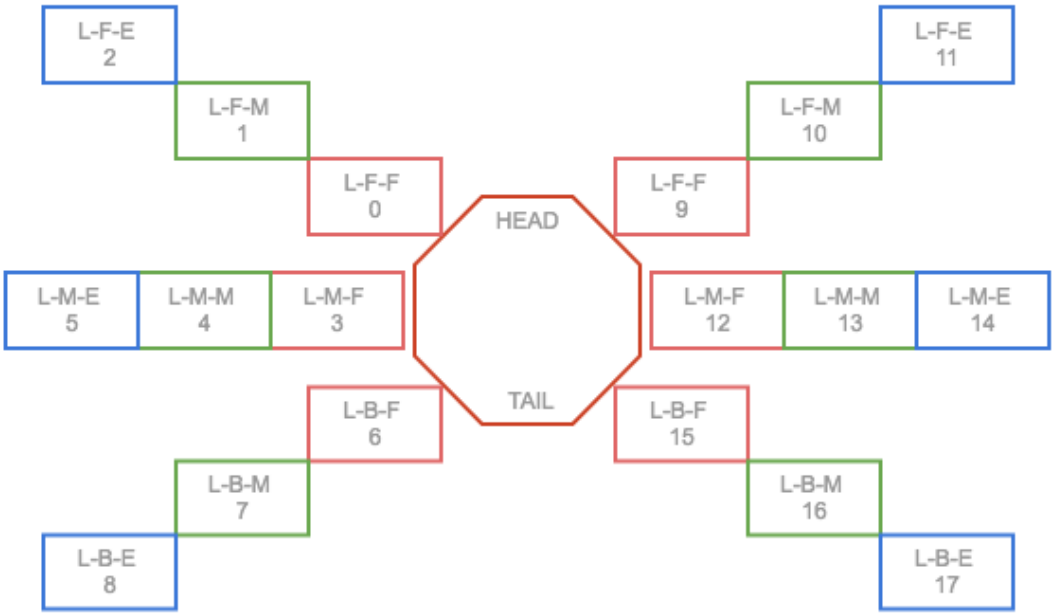

图 3-2 舵机名称与 index 映射表

3.2 软件平台架构

六足机器人嵌入式系统基于 FreeRTOS 系统进行开发，每一个模块以独立的线程工作，分为健康检查线程，动作执行线程，舵机控制线程，蓝牙帧处理线程。健康检查线程的优先级最高，舵机控制的线程优先级最低。蓝牙帧处理，动作执行，舵机控制这三个线程都一直处于休眠状态，直到它们等待的信号量到来时，才会被唤醒。

从软件层次上来说，系统分为 5 层（软件平台架构如图 3-3 所示），最底层为 ST 公司提供的 STM32F1x 底层驱动库，这一层主要实现底层的硬件操作，例如寄存器的读写，I/O 口的初始化和配置等工作。第二层为依赖于硬件平台的模块驱动层，实现如蓝牙 4.0 模块的初始化操作，读写操作，中断处理等，这

一层需要根据不同的开发板，开发不同的驱动。FreeRTOS 系统则建立在模块驱动层之上，FreeRTOS 可以提供线程的创建，调度，信号量，互斥锁，邮箱等基本的操作系统的操作。而 ARM 公司开发的 CMSIS-RTOS 是一款统一各个实时嵌入式操作系统的更抽象的接口的操作系统。CMSIS-RTOS 严格意义上不算是一个独立的嵌入式操作系统，而是需要依赖于其他嵌入式系统的一个抽象层，为上层的应用程序提供更友好，更直观的接口，同时 CMSIS-RTOS 隐藏了各大嵌入式操作的接口的差异，从而可以更快的从一个嵌入式操作系统，移植到另外一个嵌入式操作系统。但是，目前 CMSIS-RTOS 支持的嵌入式操作系统仍然比较有限，其中使用最广泛的还是属于 FreeRTOS。

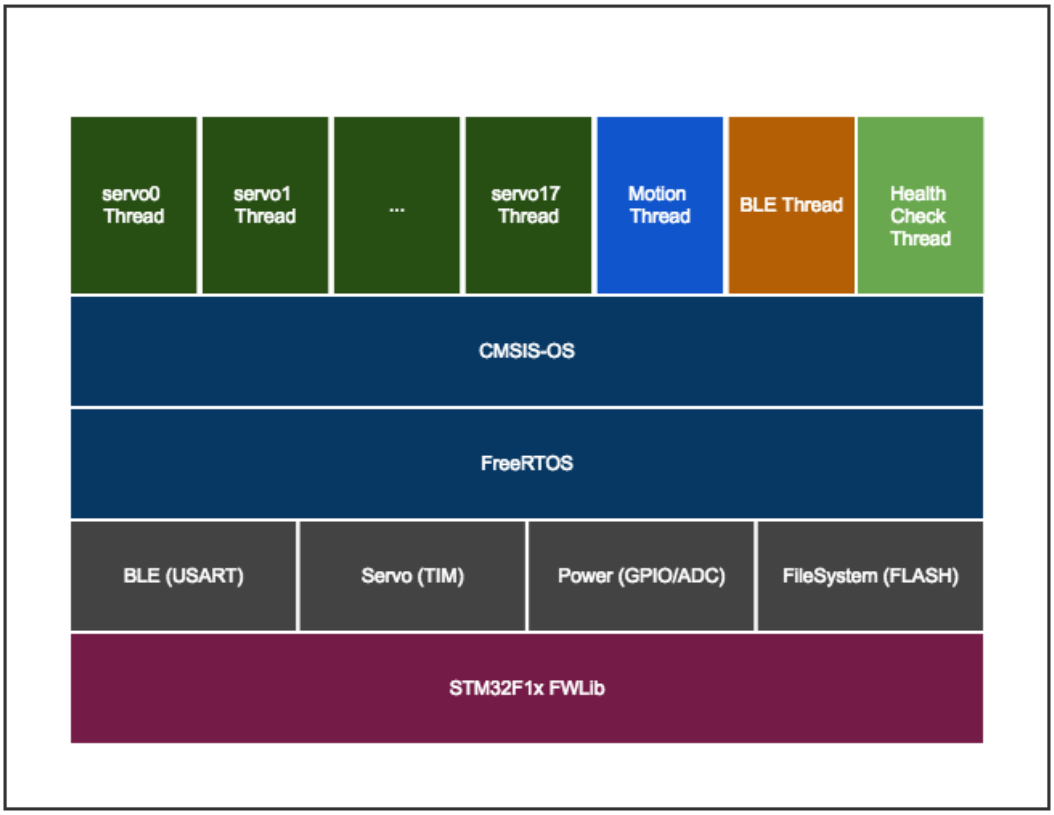

图 3-3 软件平台架构图

CMSIS-RTOS 提供 FreeRTOS 没有的一些操作系统函数，例如 Signal 二元信号，邮箱队列。

应用层根据需要，分为四类线程，第一是需要实时性很高的健康检查线程，实现电压和电流的 ADC 采样。实时优先级很高，是需要在电流过大的时候及时的切断开关电源，防止烧毁舵机。同时电压采样需要在电压过低的情况下，也

及时切断电源，防止电压过低造成电池寿命减少。同时，健康检查线程需要根据六足机器人当前的状态，即使反馈信息给用户，例如当 App 接入后，通过用户 LED 灯的灯闪频率，告知用户，当前已接入 App。或者出现电池电压过低，电流过载的情况下，即使打开蜂鸣器，以预先定义的频率发出声音，告知用户有异常的发生。

同时系统为每个舵机建立一个线程，因为在执行动作的过程中，某些舵机需要同时转动角度，而采用每个舵机独立线程的方式，可以保证每个舵机的线程都有机会得到一个时间片，从而执行转动操作。

动作线程是根据 App 发送得指令，以动作帧的形式执行操作，动作线程在需要转动舵机的情况下，主动发送一个信号 signal 给指定的舵机线程，从而唤醒该线程执行转动操作。动作线程除了唤醒舵机线程执行转动操作外，也可以执行其他命令，例如等待命令，设置速度命令，循环帧指令。

蓝牙通信线程是用于通过蓝牙模块接受和发送数据帧。蓝牙通信线程也是一直处于休眠状态，当蓝牙模块接受到数据，立刻发送一个信号 signal 给蓝牙通信线程，唤醒该线程，并对数据进行处理。数据首先需要校验是否有错，在校验正确的情况下，根据不同的请求头部，执行不同的操作，例如，获取当前的健康状态报告，获取文件系统中动作文件列表。

控制六足机器人的 App 是基于 Apple 的 iOS 平台开发的应用程序，名为 RobotScratch 。RobotScratch 架构如图 3-4：

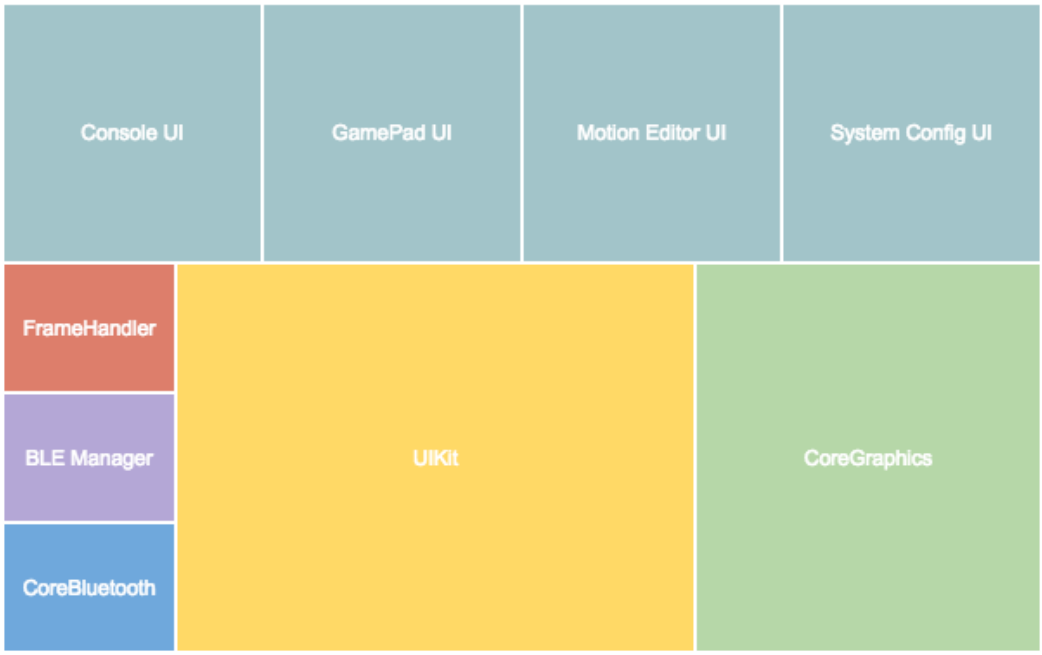

图 3-4 RobotScratch 架构图

3.3 Objective-C 的介绍

Objective-C (简称 OC)是 Brad Cox 基于 C 语言，并运用了 SmallTalk 语言消息机制，而设计的一种面向对象的语言[10]。Apple 苹果公司后来采用了 OC，并在很长的一段时间内，把 OC 作为 OSX 和 iOS 的主要开发语言，直到 2014 年 Swift 的出现。OC 以 GNU Compiler Collection (GCC)[11]或者 Clang 作为编译器，后来 Apple 苹果公司放弃使用 GCC，并在其开发环境 Xcode 中使用 LLVM 替代 GCC。

OC 可以说是 C 的一种扩展，或称之为 C 语言的超集。OC 完全与 C 语言兼容，同时可以很容易的调用 C 语言的库，实现对 C 语言现有代码对重用。

OC 相对于传统的 C 语言，加入了如下扩展语法：

（1）消息 Message。作为一种面向对象语言，与对象的交互尤为重要，而 OC 采用的是消息传递的机制，而不是使用方法调用的模式。在 OC 中，方法是绑定到对象某个特定的代码块中，在运行过程中，当对象接收到消息后，才动

态解析该方法，得益于这个特性，开发者可以给对象动态地绑定方法，十分灵活。

（2）协议 Protocol。协议是一种对象必须/可选地实现规定的方法的一种机制。对象遵循该协议后，必须要实现该协议中所有 required 的方法，并可选的实现 optional 的方法[12]。

（3）类别 Category。类别是一种代码的组织方法或者是对已有代码对的一种扩展。通过类别，开发者可以方便的对给定的类添加自定义的方法，从而实现对已有类的扩展。

（4）代码块 Block。代码块类似于其他一些动态语言的 lambda 表达式或者 Closure 闭包。代码块被广泛应用于 iOS 和 OS X 的开发当中，提供代码的异步执行[13]。

3.4 iOS 客户端使用的 framework

RobotScratch 使用了 iOS 平台的如下 framework：

（1）CoreBluetooth，CoreBluetooth 是在 iOS5 首次引入的，基于蓝牙 4.0 规范，硬件需要 4s 以上集成蓝牙 4.0 模块的 iPhone。CoreBluetooth 主要是基于 delegate 代理的方式组织代码，因为蓝牙的通信设计到异步通信，同时需要在线程中切换。所以 RobotScratch 在 CoreBluetooth 之上建立了另外一个为上层 UI 提供基于 block 异步调用的蓝牙服务层。

（2）UIKit，UIKit 是 iOS 平台最基本的框架之一，所有涉及到界面 UI 的代码都需要用到 UIKit。

（3）CoreGraphics，CoreGraphics 是提供给开发者的一个轻量级的 2D 图形渲染库[14]。通过 CoreGraphics，开发者可以实现例如基于路径的图形绘制，图形转换，颜色管理，渐变和阴影效果[15]。

（4）ReactiveCocoa，ReactiveCocoa 是一个由 Github 开源的响应式和函数式编程相结合的框架。通过 ReactiveCocoa，开发者可以实现 MVVM (Model – View – View Model)的模式，并以松散的结构来组织代码。而且代码会具有更好的测试性。

3.5 iOS app 架构

RobotScratch 架构采用 MVC 模式组织代码：

（1）Model 数据层：数据层是对六足机器人的数据交互对象进行封装。例如蓝牙帧对象，动作帧对象，健康数据对象等。

（2）蓝牙操作封装层（BLE Manager）：蓝牙操作层是建立于 CoreBluetooth 之上，对其进行封装，并以代码块回调的方式组织代码。因为代理 delegate 的模式对于异步操作并不是那么友好，代码变得零散化，而使用代码块 block 的方式，可以让代码聚合度更高。同时蓝牙操作层封装了蓝牙 CentralManager 和 PeripheralManager，对上层应用以直观的读写的方式提供 API。简化上层应用的编程复杂度。蓝牙操作层以二进制 NSData 的方式与 CoreBluetooth 交互。

（3）六足机器人交互层（Frame Handler）：帧处理层为 UI 层提供对六足机器人的数据交互 API。该层会调用底层的蓝牙操作封装层的 API，实现蓝牙数据的传送。六足机器人交互层通过 model 数据层与上层 UI 作交互。而数据对象需转换为 NSData 二进制数据后，再调用下层的蓝牙交互层。

（4）UI 层：UI 层又分为 Controller 视图控制器和 View 视图层。主要完成与 mode 数据层的交互，保存。同时需要调用下层的六足机器人交互层的 API，完成与六足机器人的通信。

第 4 章 基于 iOS 可视化操作的六足机器人的下位机系统实现

六足机器人下位机采用基于 FreeRTOS 实时操作系统进行开发，代码分层实现功能。系统总体分为以下三个层次：

（1）HAL 硬件驱动层

硬件驱动层实现对硬件的抽象编程实现。隐藏底层硬件的具体操作，只给上层应用暴露必要的操作函数。硬件驱动层直接与 STM32F1x 的官方固件驱动交互，可以减少自己配置寄存器的麻烦，提高程序的稳定性。

（2）应用服务层

应用服务层是建立与硬件驱动层之上，对硬件的操作做进一步的抽象，并实现更多的底层细节隐藏。例如蓝牙模块是基于 USART 串口作为底层通信，硬件驱动层实现蓝牙的读写功能，而应用服务层则调用硬件驱动层的函数，实现上层的蓝牙帧处理操作。

（3）系统应用线程

系统应用线程则是系统主函数中为每个独立的模块创建的线程。各个线程之间独立运行，统一由 FreeRTOS 调度。每个线程都有个字的优先级别。而线程之间通过操作系统的系统调用，实现通信，例如信号 Signal，互斥锁 Mutex，信号量 Semaphore，邮箱 Mailbox。

4.1 HAL 硬件驱动层

硬件实现了开发板模块的底层硬件驱动，其中底层驱动包括：

4.1.1 舵机驱动

舵机驱动基于 TIM 定时器 PWM 实现对舵机的控制[16]。由于六足机器人有 18 个舵机，每个舵机需要一路独立的 PWM 控制，STM32F103RC 拥有 8 个定时器，其中 TIM1 和 TIM8 为高级定时器，TIM2 到 TIM5 为通用定时器，TIM6 和 TIM7 为基本定时器。每个定时器的功能分配如表 4-1 所示：

表 4-1 定时器的功能分配

定时器	通道	功能描述
TIM8	Compare2	L-F-F 舵机 PWM 输出
TIM8	Compare1	L-F-M 舵机 PWM 输出
TIM2	Compare4	L-F-E 舵机 PWM 输出
TIM1	Compare1	L-M-F 舵机 PWM 输出
TIM8	Compare4	L-M-M 舵机 PWM 输出
TIM8	Compare3	L-M-E 舵机 PWM 输出
TIM1	Compare4	L-B-F 舵机 PWM 输出
TIM1	Compare3	L-B-M 舵机 PWM 输出
TIM1	Compare2	L-B-E 舵机 PWM 输出
TIM3	Compare3	R-F-F 舵机 PWM 输出
TIM3	Compare4	R-F-M 舵机 PWM 输出
TIM2	Compare3	R-F-E 舵机 PWM 输出
TIM5	Compare4	R-M-F 舵机 PWM 输出
TIM3	Compare1	R-M-M 舵机 PWM 输出
TIM3	Compare2	R-M-E 舵机 PWM 输出
TIM5	Compare1	R-B-F 舵机 PWM 输出
TIM5	Compare2	R-B-M 舵机 PWM 输出
TIM5	Compare3	R-B-E 舵机 PWM 输出
TIM6	NA	定时器采样
TIM7	NA	系统状态展示定时器

4.1.2 蓝牙驱动

本项目采用基于 CC2540 的蓝牙 4.0 透传模块，支持 AT 指令配置，蓝牙外设与主机连接后，实现透明的数据传输，所有操作都通过串口完成，驱动层无需考虑复杂的蓝牙协议实现。对应少量的数据传输，非常方便。

蓝牙驱动层实现了蓝牙的初始化，蓝牙的接收中断，蓝牙的数据发送功能。蓝牙的初始化流程如图 4-1 所示：

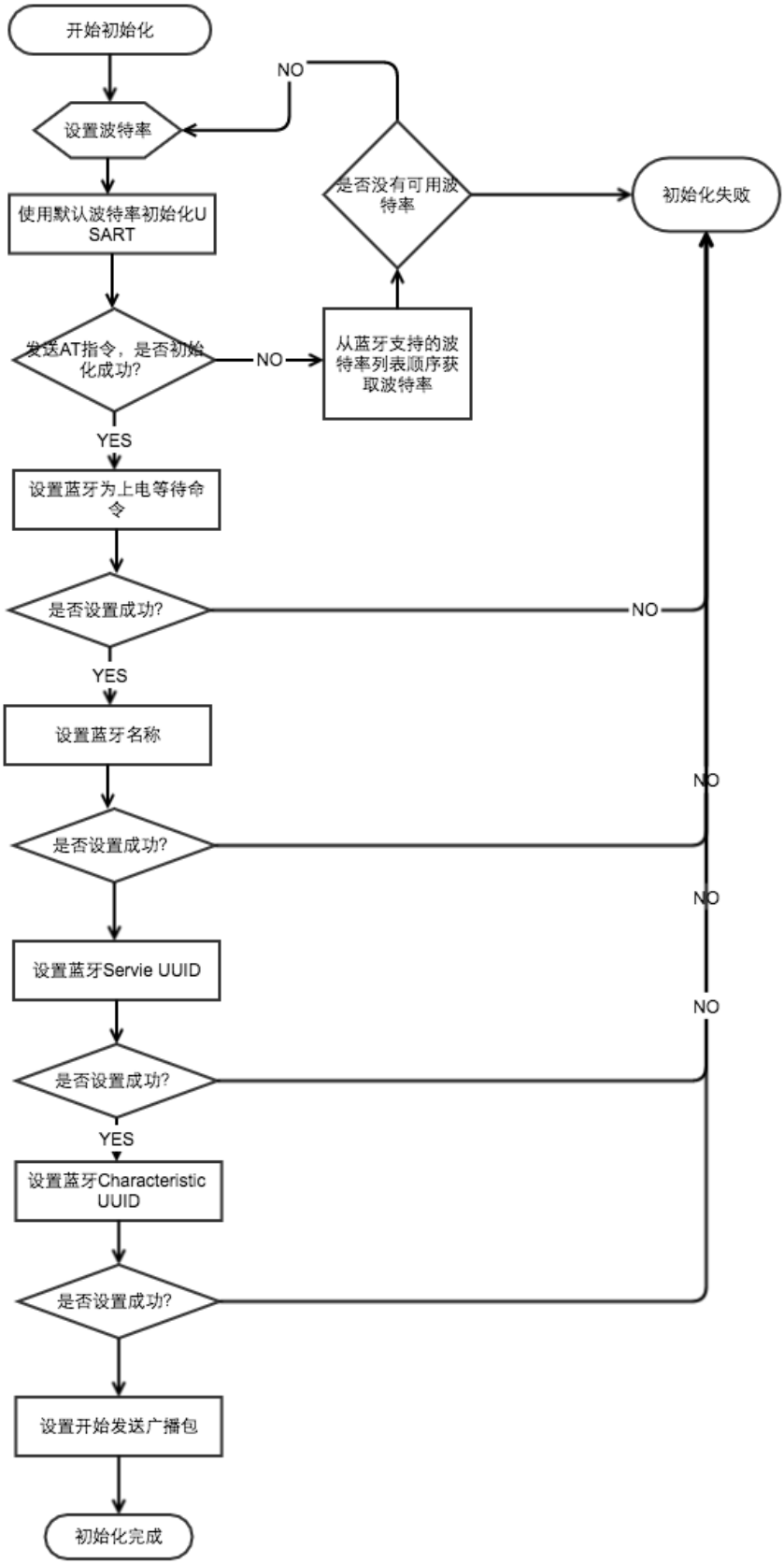

图 4-1 串口蓝牙初始化流程图

蓝牙配置大概完成如下步骤：

（1）串口波特率配置，蓝牙自带配置保存功能，所以波特率配置只需首次配置即可，后续上电，每次按默认波特率配置，减少蓝牙对内部 flash 写入的频率，而且可以提高上电初始化的速度。

（2）蓝牙上电等待命令配置。蓝牙模块默认是上电立刻等待主机接入，而这往往会导致设备未配置完成，而已经有主机接入该设备。所以必须设置蓝牙未上电等待命令。

（3）蓝牙名称的配置。蓝牙名称将会随广播包一起发送，此名称为人类友好的字符串名称，方便用户识别。

（4）配置蓝牙的 Service UUID 和 Characteristic UUID。对于 BLE 蓝牙而言，每个蓝牙都会提供至少一个服务 Service，而每个服务还可以包括多个特性 Characteristic[17]。服务与特性以 UUID 全球唯一编号的一个字符串作为该服务或属性的标识，用于区分不同的服务或属性。每个特性可以根据实际需要设置未可读，可写，可读写，可以订阅等状态。特性 Characteristic 是主机与设备通信的基本单元。所以在上电后，需要配置蓝牙的 Service UUID 和 Characteristic UUID。

（5）完成配置后，即可以设置蓝牙开始广播数据包。此时蓝牙的配置已完成。

蓝牙模块以串口作为底层与 MCU 之间的通信接口，当蓝牙每接收到接命令帧的一个字节后，都会触发串口的中断，串口中断需要知道何时是一个完整的数据帧。所以本项目规定，命令帧的总长度为 6 字节。当串口接收到 6 字节的数据后，通过 signal 的形式唤醒蓝牙处理帧线程，从而达到通知上层应用的功能。

4.1.3 ADC 采样驱动

STM32F103RC 附带两个 ADC，每个 ADC 可以支持 12bit 的模数转换精度。T74 硬件实现了电压和电流的采样电路，可以方便的通过软件 ADC 直接读取采样值，并进行后续的滤波和平滑处理。

4.1.4 LED/蜂鸣器驱动

开发板附带一个用户 LED 灯和一个蜂鸣器，用户可以根据需要点亮 LED 或打开蜂鸣器实现六足机器人的状态的呈现。

4.1.5 Flash 读写驱动

STM32F103RC 板载 256KB FLASH，每一页的大小为 2KB。为了充分利用板载 FLASH 的空间，同时不同区域需要存储不同的数据，所以 FLASH 区域划分如表 4-2 所示：

表 4-2 FLASH 区域划分

区域名称	大小	作用描述
Bootloader	18KB	Bootloader 区域用于存储引导程序
Bootloader Data area	2KB	Bootloader data area 存储引导程序相关的用户数据
PGM	64KB	用户程序区域
PGM Upgrade area	64KB	用户程序升级区域
System Config	2KB	系统配置数据
System Motion area	40KB	系统动作存储区域
User motion area	40KB	用户动作存储区域
User Data	26KB	用户预留数据区域

FLASH 传统的写入过程为：

（1）解锁 flash；

（2）以页为单位擦除数据；

（3）完成用户数据的写入；

（4）最后对 flash 上锁完成数据的固化操作。

但是这样的操作有非常大的局限性，每次写入必须保证写入的数据已准备完全。但是用户实际操作过程中，往往无法一次准备好所有数据，例如分片数据的连续存储，所以，本项目借鉴了数据库的事务概念，把一个写操作切分为

多个步骤来完成，从而方便用户自由的选择何时开启一个事务，何时写入数据，何时提交事务。Flash 事务操作如图 4-2 所示：

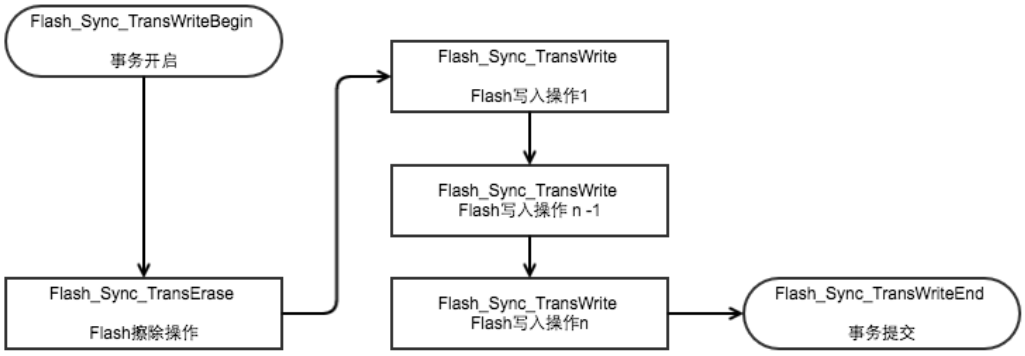

图 4-2 Flash 事务操作

FLASH 驱动分为如下几个函数

（1）Flash_Sync_Write 同步方式写入 flash，该方法自动调用 Flash 擦除函数，并解锁 flash，写入完毕后，对 flash 上锁。此函数适合一次完整的数据写入。

（2）Flash_Sync_TransWriteBeginFlash 事务开启操作，完成对 flash 的解锁

（3）Flash_Sync_TransErase Flash 擦除操作，对指定的数据区域进行擦除

（4）Flash_Sync_TransWrite Flash 事务写入操作，每个事务可以包含 1 个或多个写入操作，写入操作以字节 byte 的形式进行

（5）Flash_Sync_TransWriteEnd Flash 解锁：Flash_Sync_TransWriteEnd 必须与 Flash_Sync_TransWriteBegin 成对出现。

4.1.6 定时器驱动

定时器 7 用作系统状态和错误码展示。当系统出现如下异常时，定时器定时触发 LED 灯闪和蜂鸣器的开关。用于提示用户系统出现异常情况。

4.2 应用服务层

4.2.1 蓝牙帧处理 RFHandler

六足机器人与 app 之间的通信，分为主从关系。iPhone app 为主（master），六足机器人为从（slave）。通信永远由 app 发起。六足机器人被动接收消息。

当蓝牙模块接收到一个完整的帧之后，首先必须校验该帧是否合法，校验的计算公式如下：

$$Checksum = ((D_0 + D_1 + D_2 + \cdots + D_n) \,\&\, 0xFF\,) \wedge 0xFF$$

应用层蓝牙帧（后文简称蓝牙帧）分为命令帧，命令答复帧，批量数据帧，数据答复帧这 4 中类型。通信模式分为命令收发模式，批量数据传输模式两种，这两种模式由以上的 4 种帧类型组合完成。

命令收发模式过程如下：

主机 App 发送一个命令帧，命令帧结构如表所示，命令帧分为帧头，帧负载和帧校验和。帧头用于指定本命令帧的请求类型。

从机(六足机器人)根据不同的命令，有选择性的发送命令答复帧。命令答复帧如果遇到未知的命令，则发送 RFFrameTypeInvalidRsp (未知命令) 为帧头的答复，未知命令答复格式如表 4-3 所示：

表 4-3 未知命令答复格式

帧头 (1 字节)	未知命令帧的帧头 (1 字节)	填充值 (3 字节)	校验和 (1 字节)
0x00	0xef	0x00 0x000x00	0x10

六足机器人下位机根据不同的命令，执行不同的命令，对于无数据负载的命令答复，格式如表 4-4 所示：

表 4-4 无数据负载的命令答复

帧头 (1 字节)	请求的命令帧头 (1 字节)	填充值 (3 字节)	校验和 (1 字节)
0x0E	0x03	0x00 0x000x00	0xEE

如果命令请求的答复包括数据负载，则适用表 4-5 所示格式：

表 4-5 数据负载的命令答复

帧头 (1 字节)	数据负载 (最大为缓冲区最大长度 - 2 字节)	校验和 (1 字节)
0x0E	0x01 0x02 0x03 …	0x

批量数据传输模式适用于主机需要发送大量数据（长度超过命令帧帧长）。因为从机无法得知主机需要发送的数据有多长，所以批量数据帧在传送前，需要发送一个批量数据帧帧长的命令帧，以告知从机，接下来主机需要发送多长的数据。蓝牙通信双方交互流程如图 4-3 所示：

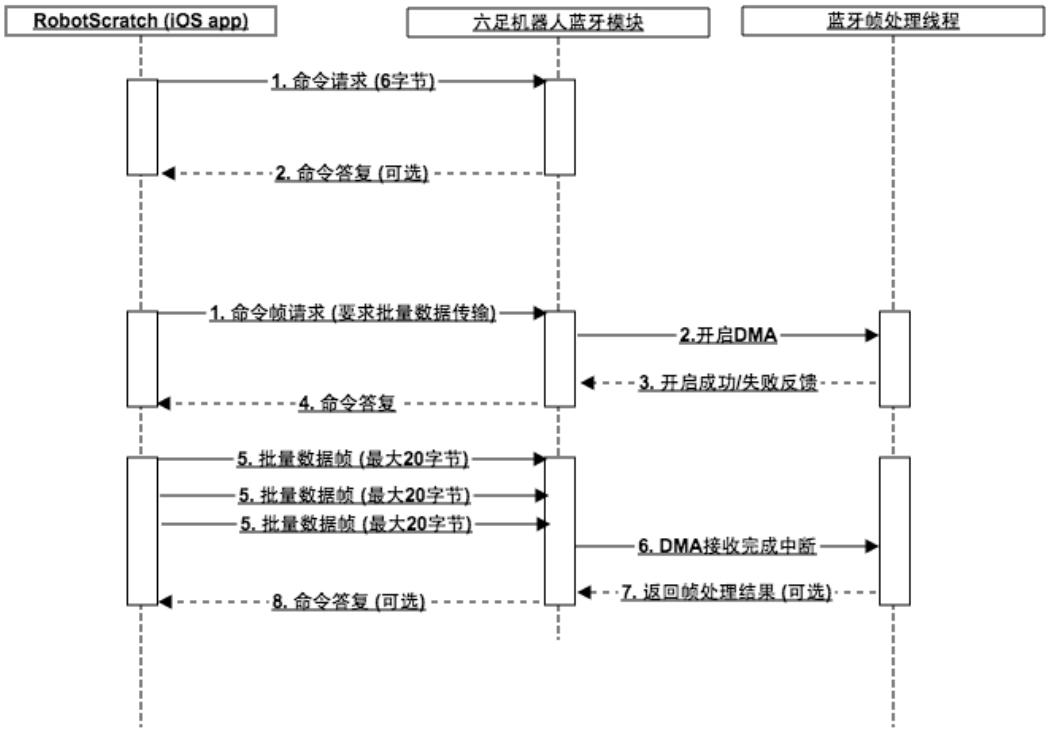

图 4-3 蓝牙命令/数据帧交互图

批量数据传输模式流程如下：

（1）主机发送 RFFrameTypeNextReq 命令帧，命令帧的数据负载前两个字节为下一帧批量数据数据帧的整帧长度。

（2）从机收到该命令帧后，设置 USART 串口为 DMA 中断，并配置接收到的批量数据帧的长度为 DMA 完成中断的数据长度，并开始监听数据。

（3）主机发送批量数据帧，当所有数据传送完毕后，从机 DMA 自动发生中断，并调用数据帧校验，并执行该帧对应的指令操作。

（4）从机根据不同的帧命令的定义，决定是否发送反馈给主机。

（5）蓝牙帧根据不同的命令调用 System 系统服务模块的函数，实现不同的操作，并根据需求决定是否返回答复帧给主机。从而有效降低蓝牙模块与其他模块的偶合度。

蓝牙帧命令可以分为如下 5 类：

（1）系统配置命令帧。包括每个舵机的最小角度，最大角度，初始化角度，舵机角度微调帧。

（2）Firmware 固件升级帧。包括固件长度请求帧，固件批量数据帧，固件下载结束帧。

（3）舵机控制帧。包括摇杆控制帧，控制台舵机角度同步帧，控制台舵机角度设置帧，单个舵机角度调整帧。

（4）文件操作帧。其中包括获取系统动作帧，获取用户动作帧，下载用户动作帧，用户动作文件删除帧。

（5）其他命令帧。包括获取固件版本帧，控制模式更改帧，六足机器人重启帧，舵机电源开关帧。

蓝牙帧处理层在接收到蓝牙串口的数据中断后，立刻对串口缓冲区的数据进行处理。蓝牙数据帧分为 3 个部分，结构如表 4-6 所示：

表 4-6 应用层帧结构

功能	长度	描述
帧头 header	1 字节	帧头用于区分不同的命令请求
帧负载 payload	命令帧 4 字节 数据负载长度 = 缓冲区最大长度 - 2	实际的数据负载，对于命令帧，固定为 4 字节；对于大数据帧，长度由数据缓冲区的长度决定
帧校验和 checksum	1 字节	用于校验整个帧是否出现数据错误

六足机器人的蓝牙命令/数据帧帧头与命令解释如表 4-7 所示：

表 4-7 帧头命令枚举与命令解释

帧头 enum 名	帧头 enum 值	命令解释
RFFrameTypeInvalidRsp	0x00	未知命令答复
RFFrameTypeVerReq	0x01	获取版本请求
RFFrameTypeVerRsp	0x02	获取版本答复
RFFrameTypeJoystickSimpleReq	0x03	摇杆简单控制数据帧
RFFrameTypeNextReq	0x04	下一帧帧长请求
RFFrameTypeConsoleReq	0x05	控制台 Console 同步电机角度帧
RFFrameTypeUserBatchReq	0x06	下载用户动作数据请求
RFFrameTypePlayReq	0x08	执行动作请求
RFFrameTypeStopReq	0x09	停止动作请求
RFFrameTypeStateReq	0x0A	获取系统当前状态请求
RFFrameTypeStateRsp	0x0B	获取系统当前状态答复
RFFrameTypeServoReq	0x0C	获取系统当前舵机角度请求
RFFrameTypeServoRsp	0x0D	获取系统当前舵机角度答复
RFFrameTypeCmdRsp	0x0E	命令请求答复

RFFrameTypeSingleServoReq	0x13	单个舵机角度调整请求
RFFrameTypeSingleServoRsq	0x14	单个舵机角度调整答复
RFFrameTypeFileListReq	0x15	获取文件列表请求
RFFrameTypeFileListRsp	0x16	获取文件列表答复
RFFrameTypeFileDelReq	0x17	文件删除请求
RFFrameTypePlayInMemReq	0x18	下载动作至内存中，并执行
RFFrameTypeSetDefaultsReq	0x19	设置默认参数请求
RFFrameTypeChangeCtrlModeReq	0x1A	更改控制模式
RFFrameTypeGetInternalCodeReq	0x1B	获取内部工程代码请求
RFFrameTypeGetInternalCodeRsp	0x1C	获取内部工程代码答复
RFFrameTypeFwLenReq	0x1D	固件程序长度请求
RFFrameTypeFwDataReq	0x1E	下载程序固件数据帧
RFFrameTypeFwEndReq	0x1F	下载程序固件结束
RFFrameTypeGetServoDegreeMinReq	0x20	获取舵机允许角度最小值请求
RFFrameTypeGetServoDegreeMinRsp	0x21	获取舵机允许角度最小值答复
RFFrameTypeGetServoDegreeMaxReq	0x22	获取舵机允许角度最大值请求
RFFrameTypeGetServoDegreeMaxRsp	0x23	获取舵机允许角度最大值答复
RFFrameTypeSetServoAdjustmentReq	0x24	设置舵机微调角度请求
RFFrameTypeGetServoAdjustmentRsq	0x26	获取舵机微调角度请求

RFFrameTypeSetServoInitDegreeReq	0x27	设置舵机初始角度请求
RFFrameTypeGetServoInitDegreeReq	0x28	获取舵机初始角度请求
RFFrameTypeGetServoInitDegreeRsq	0x29	获取舵机初始角度请求
RFFrameTypeSetServoMinDegreeReq	0x2A	设置舵机最小角度请求
RFFrameTypeSetServoMaxDegreeReq	0x2B	设置舵机最大角度请求
RFFrameTypeGetServoConfigReq	0x2C	获取舵机角度配置请求
RFFrameTypeGetServoConfigRsp	0x2D	获取舵机角度配置答复
RFFrameTypePowerSwitchReq	0x2E	打开
RFFrameTypeRebootReq	0x2F	重启 robot 请求

4.2.2 动作处理单元 RBMotion

动作处理单元是六足机器人的一个核心模块。负责解析并执行用户定义的动作帧和系统预设的动作帧。

动作文件结构如图 4-4：

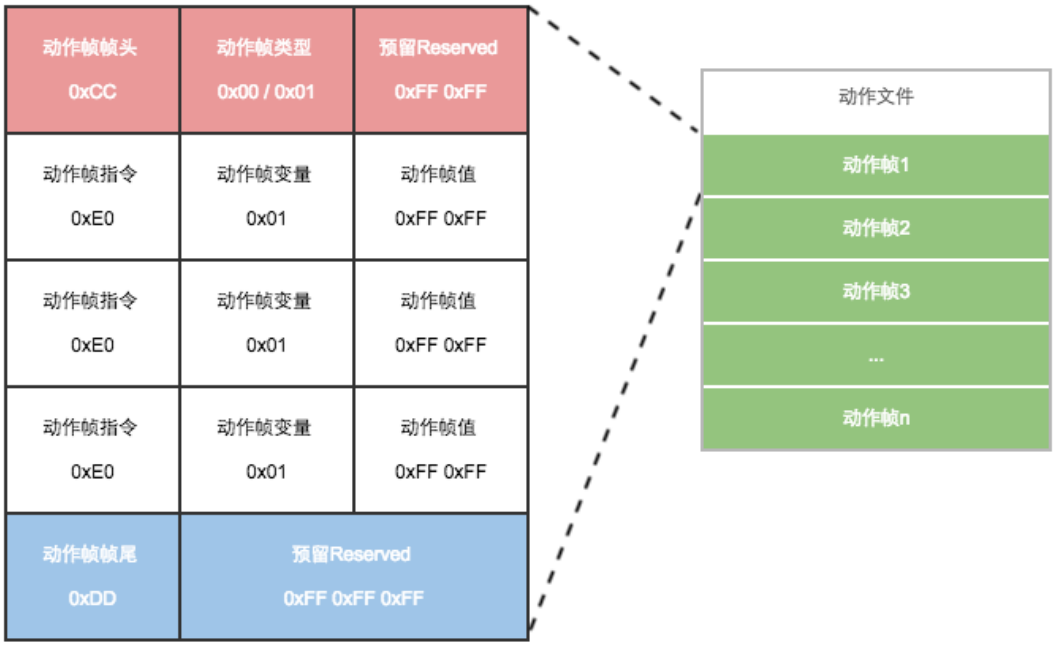

图 4-4 动作文件结构与动作帧结构示意图

动作文件由多个动作帧构成，每个动作帧定义的指令需要同一时间执行。动作处理单元顺序读取动作文件，当一个完整的动作帧读取完毕后，系统立刻执行该帧里面定义的指令。动作帧分为两类，分别为舵机转动动作帧和非转动命令动作帧，动作帧的类型由动作帧帧开头的动作帧类型字段指定。

舵机转动动作帧是完成一个或多个舵机同时转动的任务。而非转动命令动作帧则完成除了转动以外的命令，包括等待，设置速度，循环，停止。

动作帧指令如表 4-8：

表 4-8 动作帧指令

指令	变量	变量值	指令说明
Move	舵机编号 (0 – 17)	舵机角度值	对变量指定的舵机，执行转动操作，转动角度由变量值决定
Loop	NA	循环跳转帧 index	执行循环操作，循环下一帧由变量值决定
Wait	NA	等待时间	系统根据变量值定义的时间执行等待操作，等待时间单位为毫秒
Speed	NA	设置速度	根据变量值执行舵机转动速度设定

动作帧指令包括：

（1）Move 指令完成舵机的转动操作，转动的目标舵机由变量字段指定，变量取值范围为 0 - 17。每一个舵机对应一个 index 值，舵机与 index 对应值如图 1 所示。每个动作帧里可以包含 1 个或多个动作帧指令，而同一个动作帧的所有动作指令将会同时执行，通过这种方式，即可实现多个舵机同时转动的功能。

（2）Loop 指令完成动作帧的循环操作。Loop 指令所在的动作帧必须只能包含一条动作指令，并且该动作帧必须为动作文件的最后一帧。例如图 2 的动作文件，加入动作帧 n 为 Loop 循环动作帧，那么当动作单元读取完整第 n 帧后，执行跳转命令，把文件读取指针设置为 Loop 指令指定的帧，并把该帧作为下一帧，从而实现类似与编程语言的循环操作。

（3）Wait 指令。Wait 指令执行系统的等待操作，等待的时间由等待帧的动作帧值指定，等待的时间以毫秒为单位，最长等待 65536 毫秒。

（4）Speed 指令。Speed 指令执行舵机转动的速度调整操作。因为每个舵机都有最大转动速度值，所以 Speed 指令的值依赖于六足机器人的舵机参数。

对于以上指令，Loop 指令，Wait 指令和 Speed 指令都必须为动作帧的唯一一个指令，也就是该动作帧不能再包含其他指令。同时，Loop 指令必须是动作文件的最后一个帧，Wait 指令帧和 Speed 指令帧可以出现在动作文件的任意一个位置。

4.2.3 文件系统 RBFS

RBFS 是专门为六足机器人设计的文件系统。六足机器人的存储需求主要是用户编写的动作文件的存储需求，还有系统预设的动作文件。文件结构如图 4-5 所示：

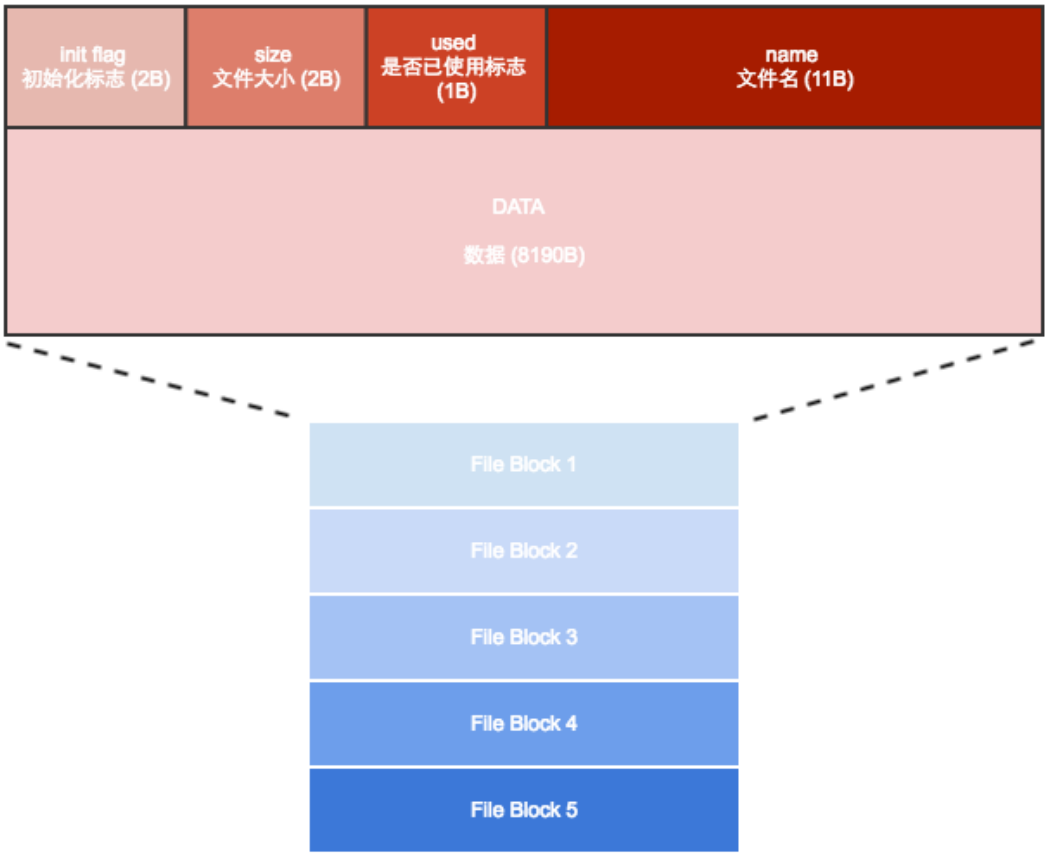

图 4-5 RBFS 文件结构示意图

RBFS 文件结构分为文件 Meta 信息，和数据区域。Meta 信息头部包含如下字段：

（1）initFlag 初始化标记，此字段用于文件系统的初始化操作，并在系统初始化后，检查每个 block 是否已初始化，若已初始化，则跳过初始化阶段；若未初始化，则为该 block 的 Meta 信息头部的字段设置对应的值。最后初始化标记改为 0x04BA，完成初始化操作。

（2）size 文件大小，此字段为 2 字节，用于标记该 block 已使用的字节长度

（3）used block 使用标记，若为 0，表示该 block 未被使用；若未 1，表示该 block 已被占用

（4）name 文件名，此字段长度未 11 字节，用于保存文件的文件名，方便用户浏览区分不同文件。

数据区域 DATA 最大长度为 8190 字节。数据区域用于存储用户或系统定义的动作帧。

RBFS 提供如下函数：

（1）RBFsTab_loadSysConfig 载入系统配置数据，系统配置数据包括舵机最小角度，最大角度，初始化角度和微调角度

（2）RBFsTab_saveSysConfig 保存系统配置数据

（3）RBFsTab_loadFsTable 载入文件系统信息

（4）RBFsTab_listFiles 列出当前的已创建的文件

（5）RBFsTab_createFile 创建新的文件

（6）RBFsTab_deleteFileByName 按文件名删除文件

（7）RBFsTab_deleteFileByIndex 按文件索引删除文件

（8）RBFsTab_updateFile 更新文件

（9）RBFsTab_readFile 读取文件到指定缓冲区

（10）RBFsTab_getFileAddressByIndex 根据索引获取文件首地址

（11）RBFsTab_getFileAddressByName 根据文件名获取文件首地址

4.2.4 系统服务 System

系统服务是一个集成所有其他底层模块的服务层，不同的模块之间调用都需要通过系统服务层，这样可以有效解除各个模块之间的偶合度，同时系统服务层还包括上电初始化的函数，状态展示等操作。

4.3 系统线程

系统初始化完毕后，创建以下 21 个线程（其中每一路舵机分配一个线程，共 18 路），分别独立实现各自的功能，它们分别是：

（1）System_HealthCheckThread 系统健康状态检查线程，检查系统是否出现异常情况，例如是否出现电流过大，电压过低的异常。系统线程同时完成状态的展示，状态的展示是以 LED 灯和蜂鸣器表达。状态展现如表 4-9 所示：

表 4-9 状态展示

状态	LED	蜂鸣器
上电初始化完毕	点亮 1 秒后，灭	响 240ms，灭
待机	点亮 0.5 秒后，灭 0.5 秒，循环	灭
App 接入	点亮 0.3 秒后，灭 0.3 秒，循环	灭
低压错误	NA	响 400ms，灭 400ms，循环
软件过流	NA	响 200ms, 灭 200ms，循环
硬件检测过流	NA	响 100ms，灭 100ms，循环

当出现异常情况，或状态改变：

（2）System_RFFrameHandleThread 蓝牙帧处理线程，当接收到蓝牙数据后，蓝牙串口中断发生，或者批量数据的 DMA 传送完毕中断发生，并发送信号 OS_RECV_APP_FRAME_BIT_4 至蓝牙帧处理线程。蓝牙帧处理过程如图 4-6：

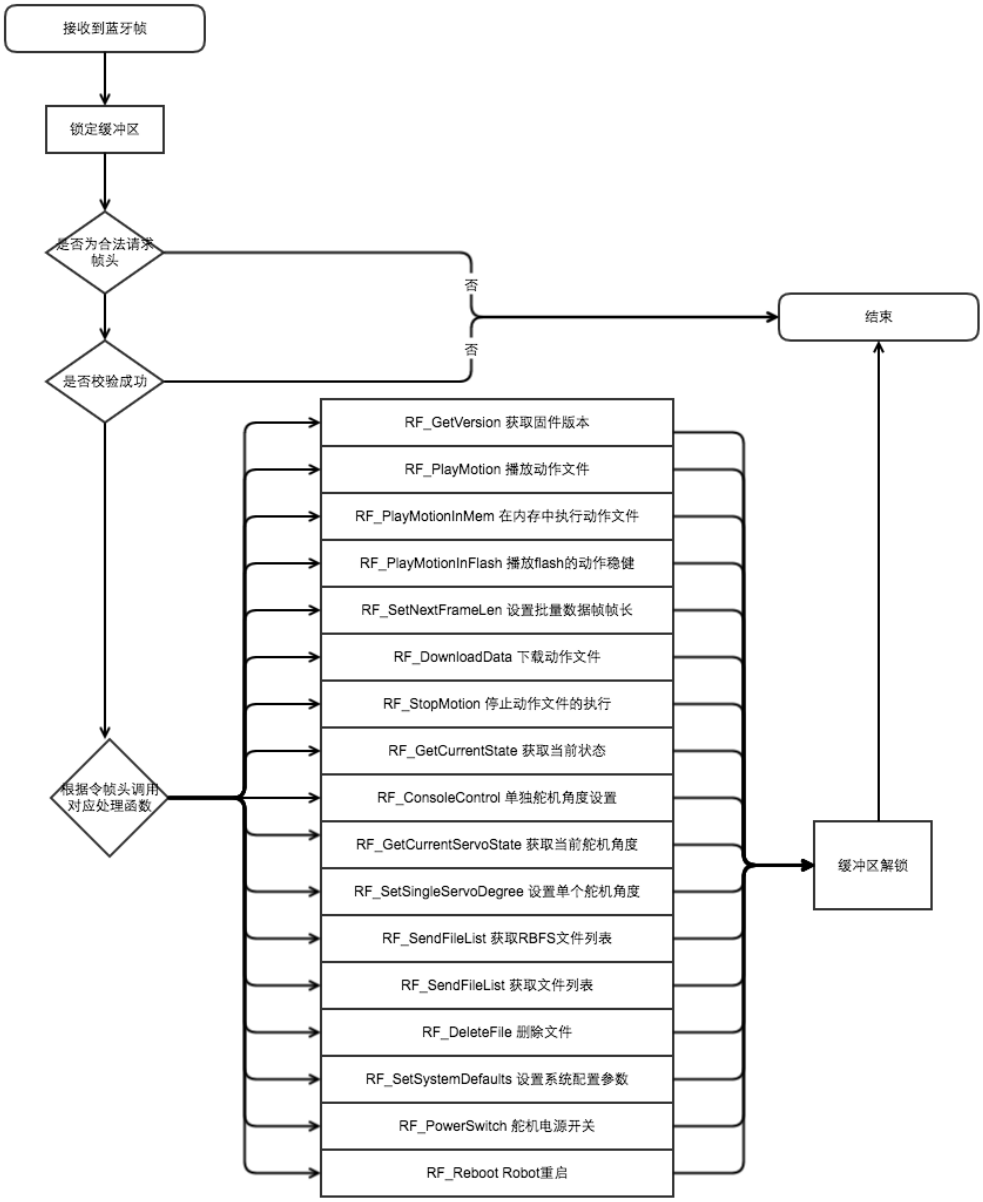

图 4-6 蓝牙帧处理线程执行流程

蓝牙帧处理线程在系统初始化完成后，自动进入阻塞状态，并等待 OS_RECV_APP_FRAME_BIT_4 signal 信号。当接收到该信号后，随即按图所示执行，详细步骤解析如下：

（1）锁定蓝牙接收缓冲区

（2）获取蓝牙接收缓冲区的首字节，判断该命令是否为合法的请求命令。若不合法则返回，并重新进入阻塞状态。

（3）计算蓝牙接收缓冲区的校验和，若校验失败，则返回，并进入阻塞状态。

（4）根据蓝牙接收缓冲区字节，判断该执行什么命令，并调用该命定对应的函数。

（5）解锁蓝牙接收缓冲区

（6）重新进入阻塞状态

System_MotionHandleThread 系统摇杆动作处理线程，该线程在系统初始化完毕后，自动进入阻塞状态，并等待 mailbox 数据 motionJoystickMailbox，该 mailbox 邮箱包括数据结构 MotionJoystickMail，该数据结构包括如下字段：

- joystickLeftXval; /*!<遥控杆控制 (左 X 轴) */
- joystickLeftYval; /*!<遥控杆控制 (左 Y 轴) */
- joystickRightXval; /*!<遥控杆控制 (右 X 轴) */
- joystickRightYval; /*!<遥控杆控制 (右 Y 轴) */

这 4 个字段均为 0-99 的整型数字，用于代表摇杆远离中心点的百分比，0 代表最左边(X 轴)或最下方(Y 轴)。99 代表最右边(X 轴)或者最上方(Y 轴)。动作单元根据 X 轴的控制值，判定机器人行走的方向，并且通过 Y 轴的控制值，判断执行动作的速度。

System_MotionServoRotateThread 系统舵机转动线程，每一路舵机单独一个线程，并且在系统初始化完毕后，自动进入阻塞状态。线程在阻塞状态中等待信号 OS_SERVO_ROTATE_BIT_7。每个线程在初始化过程中，均附带了一个代表舵机的编号的 index 参数，通过这个参数，线程处理函数可以在线程被唤醒的时候，识别到该执行哪个舵机的转动。角度数据需在释放 OS_SERVO_ROTATE_BIT_7 信号前，预先载入到角度转动预设缓冲区中。

4.4 本章小结

本章详细描述了六足机器人的下位机设计和实现方案。通过自底向上的方式分层实现一套完整的六足机器人的控制系统。包括底层的蓝牙驱动，舵机控制驱动，定时器驱动，采样驱动，flash 读写驱动，到二层的，文件系统，动作执行单元，蓝牙帧处理单元，再到第三层的基于 FreeRTOS 的动作处理线程，蓝牙处理线程，动作处理线程和系统状态展示线程。通过自底向上的方式可以有效的把底层硬件抽象为通用接口，从而减少应用层的底层操作代码。同时，降低各个但与间的耦合度，降低各个组件间的依赖。

第 5 章 基于 iOS 可视化操作的六足机器人的 app 实现

六足机器人的上位机是基于苹果公司的 iOS 平台设计和开发。六足机器人的 iOS 控制 App 名为 RobotScratch。借助 iOS 强大的图形处理功能，实现简易的六足机器人操作。同时基于蓝牙 4.0 模块通信协议，可以方便的实现 App 对六足机器人的控制。本章节将描述 App 的四大功能模块，分别阐述这四大功能模式的设计与实现方案。

5.1 RobotScratch 架构总揽

RobotScratch 遵循基本的 MVC 设计模式，App 基本分为工具类，Model 数据模型，View 视图和 Controller 视图控制器。RobotScratch UML 类图如图 5-1 所示：

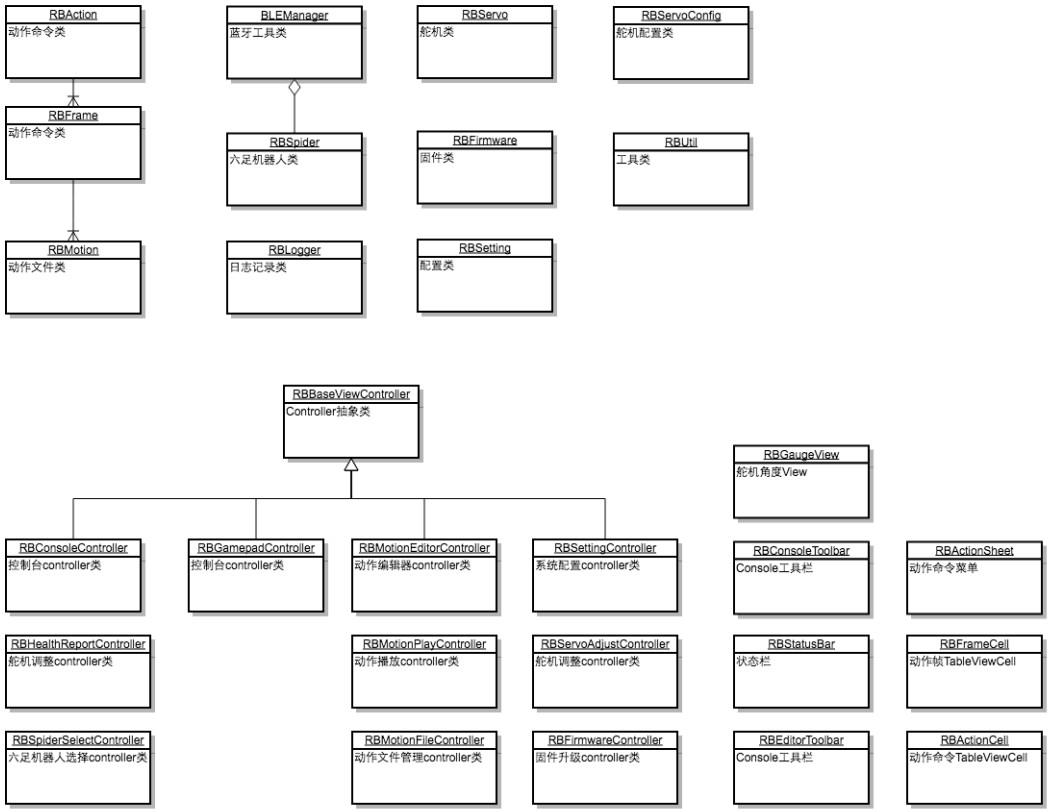

图 5-1 RobotScratch UML 类图

Model 层实现对六足机器人相关数据的一个封装，例如动作文件，动作帧，动作命令，Firmware 固件，配置信息等。

View 层包括一个视图中可重复利用的 widget 的一个抽离，并独立为一个类，例如动作命令菜单 ActionSheet，控制台的舵机角度 GaugeView。View 层的目的是把 Controller 中的独立的视图的抽离，减少 Controller 的复杂度，并尽可能实现 View 的重用。

Controller 层是整个 App 的核心，联系 Model 层与 View 层。Controller 层承担 App 大部分的业务逻辑。接下来将逐一介绍每个模块的实现方法。

5.2 iOS 蓝牙实现

5.2.1 技术背景介绍

CoreBluetooth 是 iOS 提供给用户与蓝牙 4.0 模块交互的 SDK Library。CoreBluetooth 提供两种设计模式，分别是 iOS 作为本地 Central 主机，并与远端的设备 Peripheral 交互，也可以把 iOS 作为本地设备 Peripheral，并与远端主机 Central 交互。本项目采用第一种模式，其中包括 CBCentralManager 和 CBPeripheral，分别代表蓝牙通信的主机和设备。通过 CBCentralManager，用户可以发起蓝牙链接，扫描附近的蓝牙设备，并接入指定的蓝牙设备。而 CBPeripheral 则是对蓝牙 4.0 设备的抽象，其中包括蓝牙设备的服务 Service，Service 包含多个特性 Characteristics。

CoreBluetooth 是基于代理 delegate 回调的编程模式实现与蓝牙设备的交互。CoreBluetooth 有两个重要的协议 protocol，分别是 CBCentralManagerDelegate, CBPeripheralDelegate。前者定义了主机 central 的代理方法，而后者则定义了设备的代理方法[18]。

5.2.2 蓝牙工具类 BLEManager 的设计与实现

BLEManager 使用单例模式实现对象的创建，并同时遵循 CBCentralManagerDelegate, CBPeripheralDelegate 这两个协议。BLEManager 提供的方法如表 5-1 所示：

表 5-1 BLEManager 公共方法与解释

方法	介绍
- (void)scanPeripheralsWithService:(NSString *)serviceUUIDString completion:(BLEScanCompletion)completion;	扫描外设
- (void)stopScanPeripherals	停止扫描
- (void)connectPeripheral:(BLEPeripheral *)peripheral completion:(BLEConnectCompletion)completion	连接外设
- (void)disconnectPeripheralWithcompletion:(BLEDisconnectCompletion)completion	断开外设
- (BOOL)isConnected	是否已连接外设
- (void)subscribe:(CBCharacteristic *)characteristic notifyCompletion:(BLENotifyCompletion)notifyCompletionvalueUpdateCompletion:(BLEReadCompletion)updateCompletion	订阅指定蓝牙特性
- (void)Unsubscribe:(CBCharacteristic *)characteristic notifyCompletion:(BLENotifyCompletion)notifyCompletion	取消订阅指定特性
- (void)writeValue:(NSData *)values toCharacteristic:(CBCharacteristic *)characteristic completion:(BLEWriteCompletion)completion	发送数据到指定特性
- (void)readValueFromCharacteristic:(CBCharacteristic *)characteristic completion:(BLEReadCompletion)completion	从蓝牙指定特性读取数据

BLEManager 的大部分方法使用 block 的方式，实现异步调用。通过 block 不仅让代码变得更紧凑，而且可以利用 iOS 的异步机制，实现异步处理。BLEManager 的 UML 类图如图 5-2 所示：

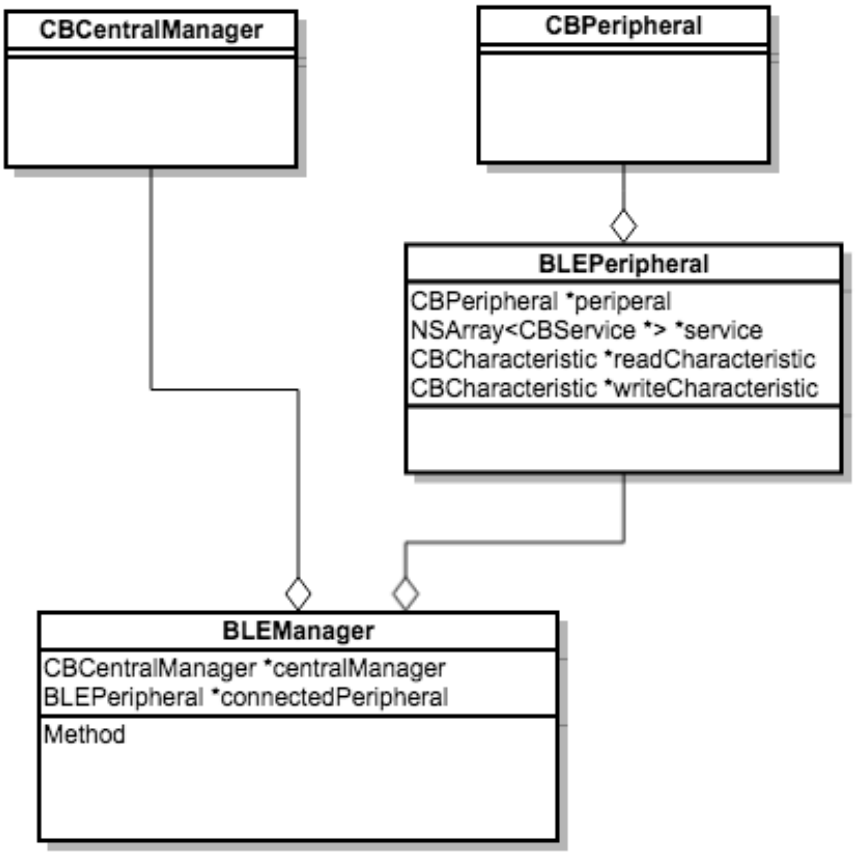

图 5-2 BLEManager 的 UML 类图

BLEManager 封装了 CBPeripheral 并创建了一个新的类 BLEPeripheral，因 CBPeripheral 的 service 在连接后的后续读写操作不可见，所以必须在连接蓝牙设备成功后，保存并引用 CBPeripheral 的 service。

BLEManager 的完整 lifecycle 如图 5-3 所示：

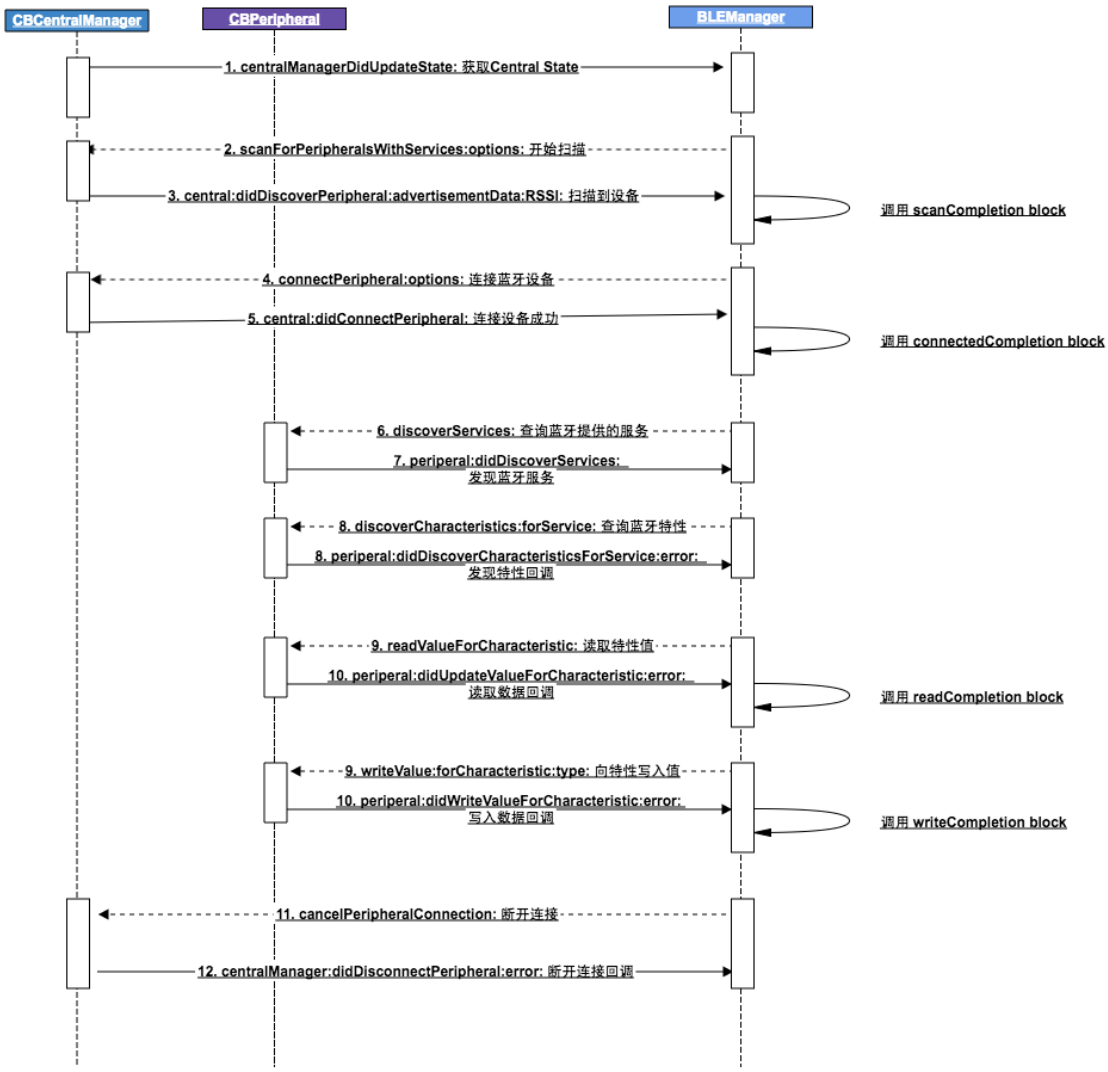

图 5-3 BLEManager 类生命周期示意图

BLEManager 生命周期如图 5-3 所，蓝牙数据交互流程如下：

（1）蓝牙设备的扫描与回调。用户可自由定义 scanCompletion 的 block 内容，实现扫描完成的回调

（2）蓝牙设备的连接与回调。用户可根据需要，定义 connectedCompletion block 的内容，实现蓝牙连接的回调。

（3）扫描蓝牙设备所提供的服务 Service。

（4）扫描蓝牙设备指定服务的特性 Characteristic。

（5）从蓝牙设备读取数据，并根据用户编写的 readCompletion block，实现读取完毕的回调。

（6）向蓝牙设备写入数据，并根据用户定义的 writeCompletion block，实现写入完毕的回调。

（7）断开蓝牙连接。

5.3 六足机器人 UI 界面的实现

5.3.1 Console 控制台的实现

Console 控制台用于给用户展示接入的六足机器人的当前状态，并提供舵机可视化操作。Console 控制台界面如图 5-4 所示：

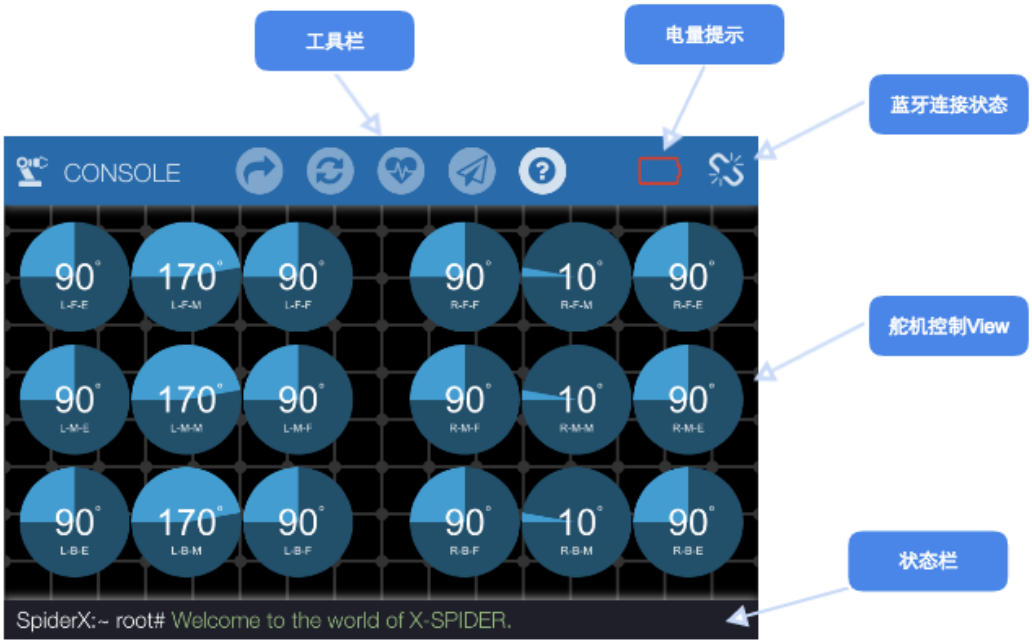

图 5-4 控制台 Console UI 组件示意图

Console 控制台分为三大区域，它们分别是：

（1）Console 工具栏，提供接入状态的展示，当前六足机器人的电量提示，发送当前的舵机角度数据至六足机器人，同步六足机器人的舵机角度，打开健康状态监测页面，和打开/关闭舵机实时调整功能。

（2）舵机角度调整区。此区域以可视化的方式展示六足机器人 18 个舵机的角度，并允许用户以拖动的形式，改变舵机的角度，亦可以双击舵机调整 View，打开单个舵机调整页面.

（3）状态栏。状态栏展示六足机器人的控制台输出信息，包括警告，错误信息。用户可以通过双击状态栏，打开 Terminal 终端，查看更详细的日志信息。

5.3.2 Gamepad 模拟遥控手柄界面的实现

Gamepad 实现类似于游戏手柄的虚拟摇杆功能，得益于 iOS 的多点触摸功能，用户可以同时控制游戏左右两个摇杆，在六足机器人已接入的前提下，发送实时摇杆位置数据至六足机器人，并实现六足机器人的方向控制。Gamepad 界面如图 5-5 所示。

Gamepad 界面分为三个区域，分别是：

（1）工具栏，可以查看当前六足机器人是否已接入，并且可以打开连接扫描页面，扫描附近的六足机器人。同时，利用 iOS 的陀螺仪 CoreMotion SDK，可以在日后的升级版本中，实现六足机器人的 PID 控制。

（2）摇杆区域，摇杆分为左右两个，左摇杆实现六足机器人的左右控制，右摇杆实现六足机器人的速度调整。

（3）状态栏，实时显示控制六足机器人的日志信息。

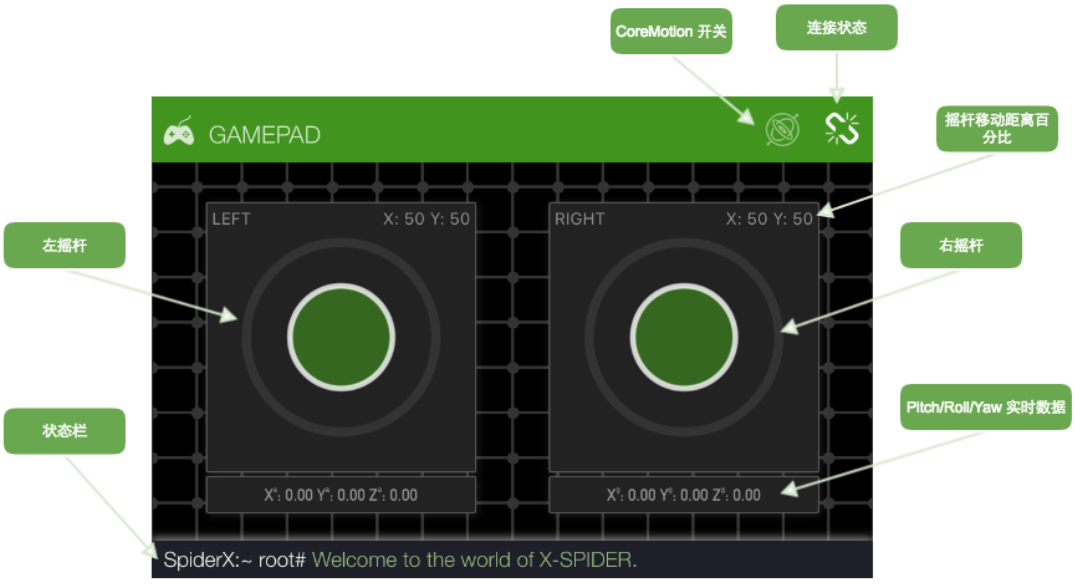

图 5-5 Gamepad 摇杆控制界面功能示意图

5.3.3 MotionEditor 动作编辑器的实现

MotionEditor 动作编辑器是 RobotScratch 最重要的一个功能，通过动作编辑器，没有编程基础的用户也可以发挥自己的创意，编写一套完整的六足机器人动作指令。

MotionEditor 的基本功能如图 5-6 所示：

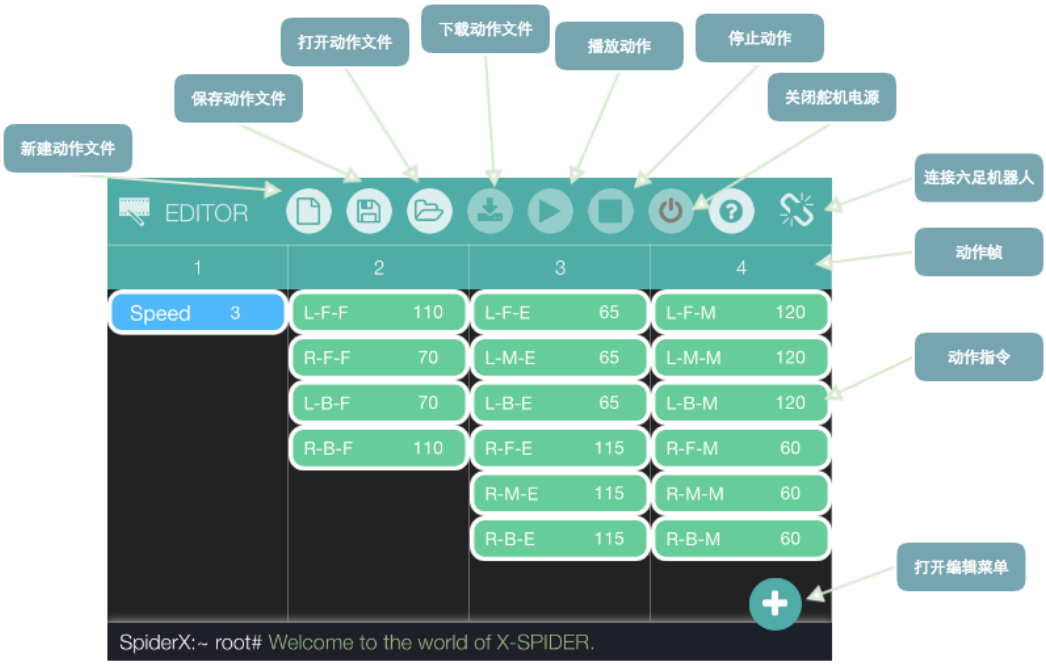

图 5-6 MotionEditor 动作编辑器功能示意图

MotionEditor 以动作文件为核心，提供动作文件的新建，编辑，保存，删除灯功能，同时支持连接远程六足机器人，获取六足机器人当前的文件列表。并支持六足机器人内置 RBFS 文件系统的文件下载，文件删除功能。

MotionEditor 的文件可以远程保存到六足机器人的文件系统中，并执行播放命令，同时也可以下载至六足机器人的内存中，执行在内存的快速播放，减少对六足机器人内部 flash 的擦写操作。

MotionEditor 动作编写区域以动作帧为单位，支持动作帧的添加删除操作，同时每个动作帧可以定义一个或多个动作命令。动作命令包括：

（1）速度设置 Speed 命令；

（2）舵机角度设置 Move 命令；

（3）等待 Wait 命令；

（4）循环 Loop 命令。

5.3.4 固件 Firmware 升级的实现

通过 RobotScratch，用户可以实现对六足机器人实现在线固件升级功能。固件升级功能如图 5-7 所示：

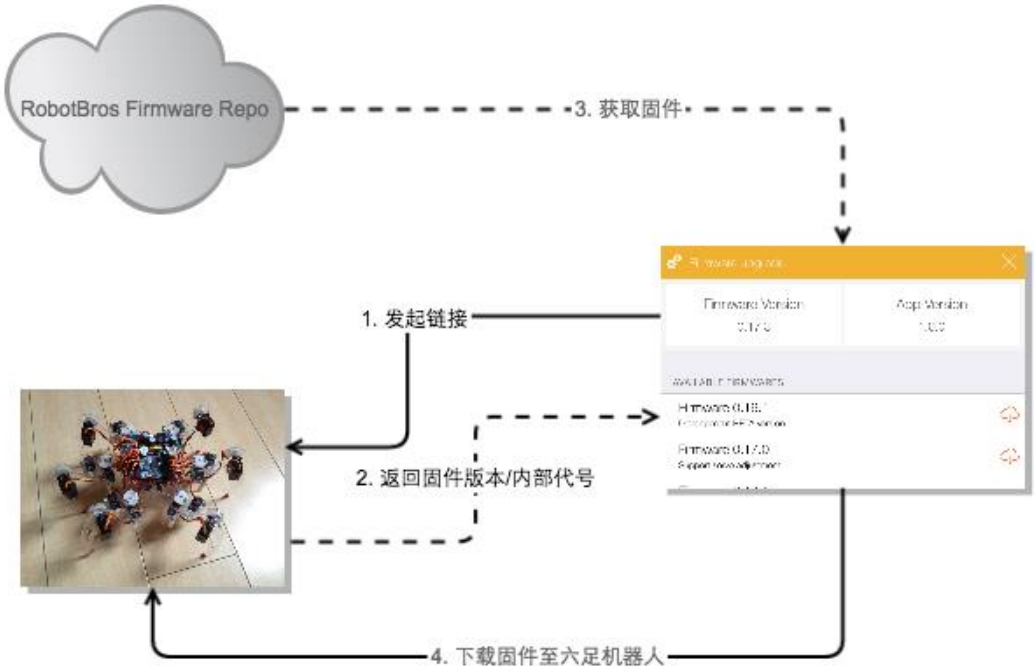

图 5-7 固件升级功能

固件升级流程如下：

（1）连接六足机器人

（2）RobotScratch 向六足机器人发送 firmware 版本请求和六足机器人 firmware 内部代号请求

（3）RobotScratch 根据 firmware 内部代号，查询云端的 Firmware Repo 固件仓库，并获取该代号对应的固件列表

（4）用户决定选择升级哪个固件

（5）RobotScratch 从云端下载对应版本的固件，并保存在本地

（6）RobotScratch 发送 RFFrameTypeFwLenReq 命令帧，告知六足机器人需要升级的 firmware 固件的文件长度，单位为字节，和该固件的 CRC32 校验和。

（7）六足机器人发送命令请求成功答复

RobotScratch 把固件文件切分为多个 18 字节的 packet，并封装为 RFFrameTypeFwDataReq 数据下载帧，最后发送至六足机器人

（8）六足机器人判断当前文件位置是否为最后一个数据包，若不是，则保存接收到的 18 个字节至 flash，若否，则通过固件长度计算最后一个数据包的数据长度，并写入该长度对应的字节至 flash。

（9）RobotScratch 发送下载结束请求 RFFrameTypeFwEndReq 命令帧，告知六足机器人下载固件结束，六足机器人计算已下载的固件的 CRC32 校验和，并与步骤 6 的 CRC32 校验和比较，若校验正确则更新 Bootloader 数据区的 Bootflag 为“Ready to upgrade”，并返回命令正确的应答帧至 RobotScratch；若校验失败，则发送命令错误帧至 RobotScratch。

（10）RobotScratch 接收到应答帧后，确定下载结束命令请求是否成功，若成功，则提示用户，需要重启六足机器人，完成 firmware 升级

（11）用户重启六足机器人，Bootloader 检查 Bootloader 数据区的 Bootflag 是否为 Ready to upgrade。若是，则标记 Bootflag 为 “upgrading”，并复制 upgrade 区的 firmware 至 program 区，最后标记 Bootflag 为“upgrade ok”。完成执行重启操作。

（12）六足机器人重启，Bootloader 检测 Bootflag 是否为“upgrade ok”，若是，则修改 Bootflag 为 normal，并尝试引导 program 区的程序，program 区的程序需在初始化完毕后，设置 Bootflag 为“normal”；如果 Bootflag 为其他值，则初始化蓝牙模块，并进入待升级模式。

Firmware 只有在下载完成，并校验正确后，才会引导该 firmware，并切 firmware 只有一次机会设置 Bootflag 为“normal”，若初始化失败，下次启动后，Bootloader 会忽略该 program，并自动进入待升级模式。这种设计可以有效保证升级的 firmware 固件是正常的情况下，才允许引导并进入该固件执行，在其他异常情况下，均进入待升级模式。

5.4 本章小结

本章阐述了基于 iOS 开发的六足机器人客户端的实现。iOS 客户端包括控制台 Console，模拟摇杆 Gamepad，动作编辑器 MotionEditor，和 app 在线升级固件的机制和实现方法。

第 6 章 基于 iOS 的可视化操作的六足机器人的集成测试

六足机器人下位机与 iOS APP 开发完毕后，需要检验该设计是否符合需求的要求。本章将阐述六足机器人下位机与 iOS App RobotScratch 配合测试的各项结果，并得出最终的测试结论。六足机器人的舵机采用银燕 EMAX ES08AII 9g 舵机，供电电池使用 7.4V 可充电锂电池，六足机器人外形框架使用亚克力板制作，iOS 端 App 安装于 iPhone 4s 之上。

6.1 蓝牙 4.0 通信集成测试

6.1.1 测试方法

iOS 客户端与六足机器人的蓝牙模块通信涉及 47 个命令帧与答复帧，测试将对这 47 个蓝牙帧对应进行逐一测试，保证测试结果符合预期要求。iOS 端使用 Xcode 作为开发调试工具，下位机采用 IAR 作为开发调试工具，二者分别在蓝牙接收发送函数中，设置断点查看每个请求帧对应的接收和发送结果，并验证接收/发送缓冲区的数据是否符合要求。

6.1.2 测试结果

获取固件版本测试结果与预期结果对比分析如图 6-1 至 6-3 所示：

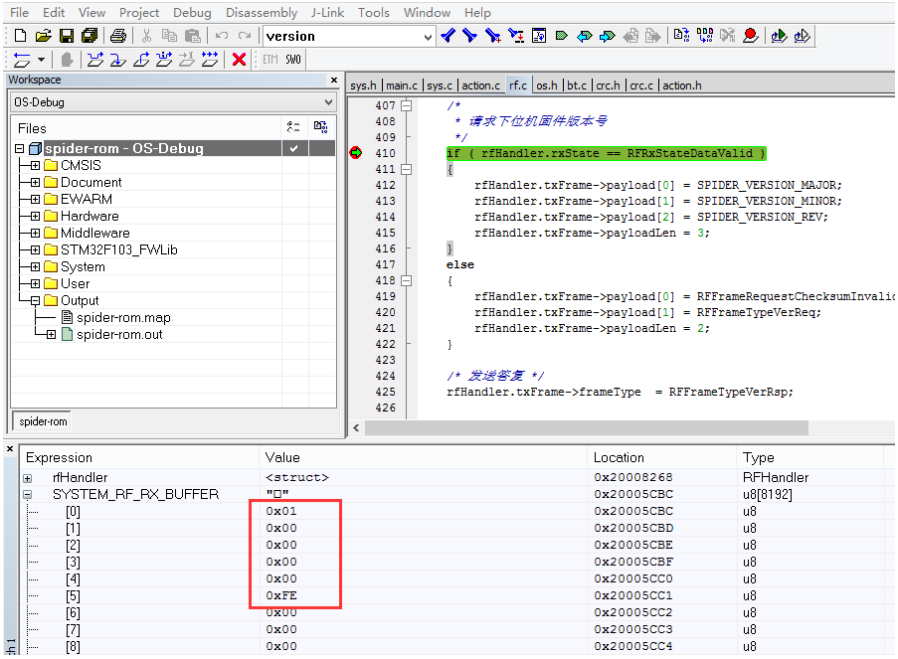

图 6-1 获取固件版本命令 IAR 接收缓冲区示意图

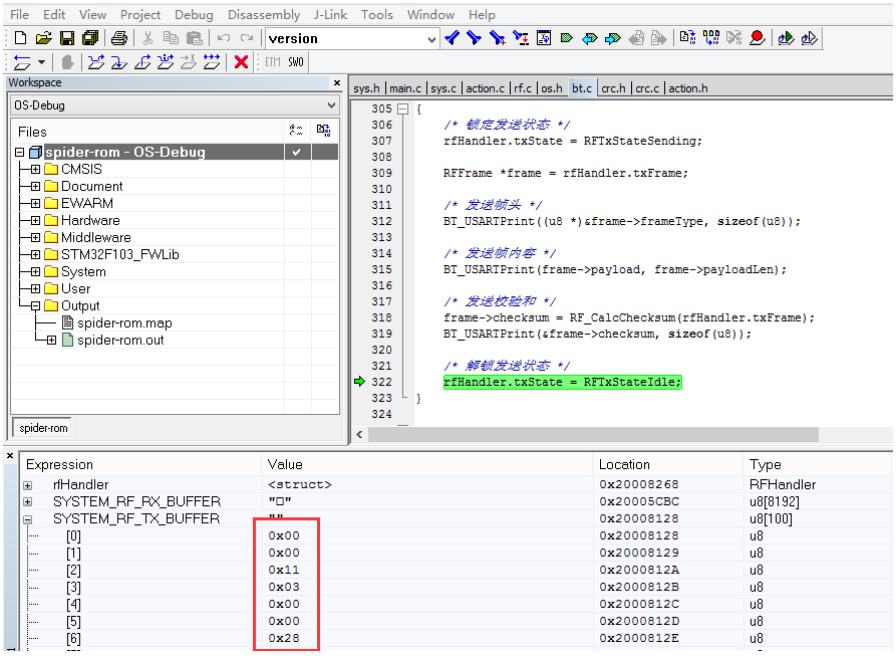

图 6-2 获取固件版本命令 IAR 发送缓冲区示意图

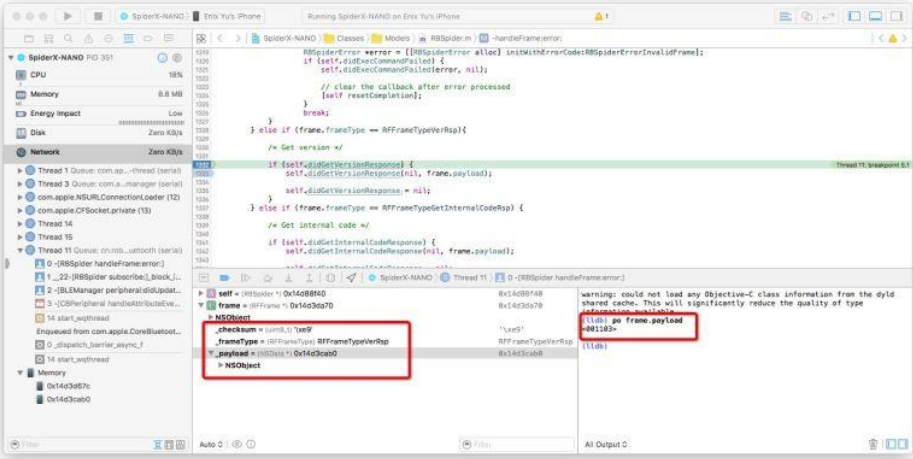

图 6-3 获取固件版本命令 Xcode 接收缓冲区示意图

根据图 6-1，当 app 发起获取固件版本的请求后，下位机捕捉到断点，蓝牙接收缓冲区如图 6-1 所示，第一个字节为 0x01，代表获取固件请求，第二至第五字节为填充字节，均为 0x0，第六个字节为校验和 0xfe，接收命令的校验正常。程序获取版本后，构造的答复帧缓冲区如图 6-2 所示，第二至第四字节分别对应版本号的 MAJOR，MINOR，PATCH，也就是 0.17.3。Xcode 获取到答复帧后，触发断点，缓冲区如图 6-3 所示，app 获取的数据跟下位机发送的结果一致。

其他命令帧亦通过这个方式执行测试，测试结果都符号要求。最后我们可以得出结论，iOS 与下位机的蓝牙帧接发都是正常的，符合需求设计要求。在确保蓝牙帧处理逻辑没问题的情况下，下一步将进行进一步的集成测试。

6.2 六足机器人固件在线升级测试

6.2.1 测试方法

为了验证升级流程，需根据如下不同的场景，测试升级流程是否正常工作。固件程序只有在正常下载，并正确设置 bootflag 后，才认为升级过程结束，如果升级过程中，任何一个步骤出错，都被认为升级失败，升级失败重启后，必须进入升级模式，等待用户下载新的固件。

（1）场景 1，六足机器人下位机只有一个 Bootloader 程序，program 区为空。下位机上电后，必须进入升级模式。

（2）场景 2，下位机有 Bootloader 程序，但 program 区只保存了部分 program 代码，固件下载过程被中断。重新启动后，bootloader 必须能重新进入升级模式。

（3）场景 3，下位机有 Bootloader 程序，并且 program 区保存了完整的 program 代码，但是 program 未设置 Bootflag 为 normal。当再次重新启动后，必须进入升级模式。

（4）场景 4，下位机已正确引导 program，并通过 iOS 平台在线对下位机进行固件升级。重启后，必须能正常引导该固件。

6.2.2 测试结果

下面将对这四个场景进行测试。

1、场景 1：

下位机擦除整个 flash，并烧写 Bootloader 程序，上电并等待 app 接入。App 进入 Setting 页面，并选择 Firmware upgrade，连接六足机器人，界面如图 6-4 所示：

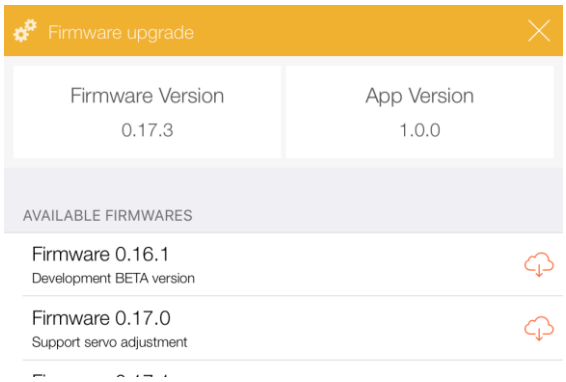

图 6-4app firmware 固件升级页面

当前下位机固件为 0.17.3，所以只能升级版本号大于 0.17.3 的固件。点击 0.17.4 的 upgrade 按钮，开始升级（如图 6-5 至 6-8 所示）：

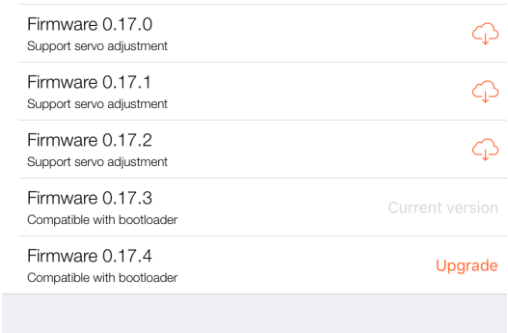

图 6-5 固件选择

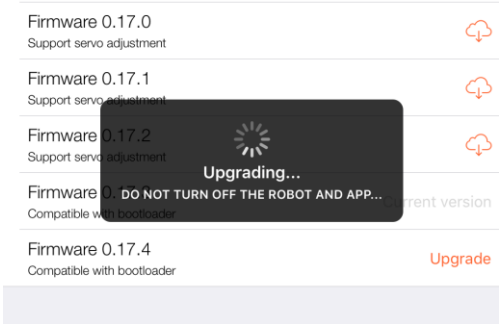

图 6-6 固件升级中

下载完毕后，重启六足机器人，完成最后的固件升级操作。

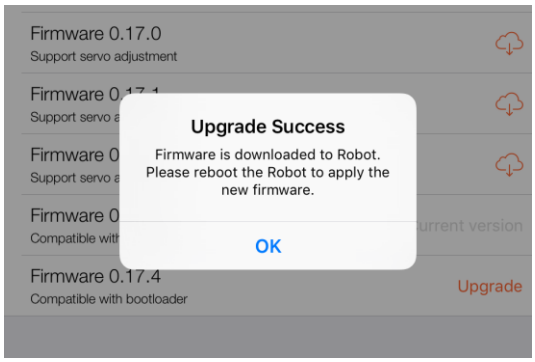

图 6-7 固件升级完毕提示

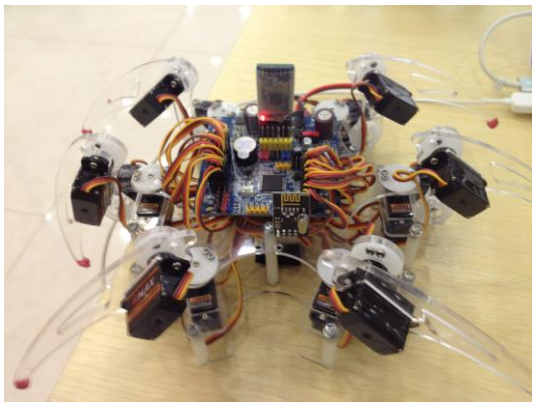

图 6-8 六足机器人重启复位

2、场景 2：

为验证构造场景 2 的条件，在 iOS 端发起下载固件请求后，关闭六足机器人的电源，从而使六足机器人的 flash 只保存了部分 firmware 固件数据，在为下载完成固件的前提下，六足机器人开机必须进入 Bootloader 模式，等待下一次用户升级固件。

测试过程如下：

（1）发起固件升级请求（图 6-9 所示）

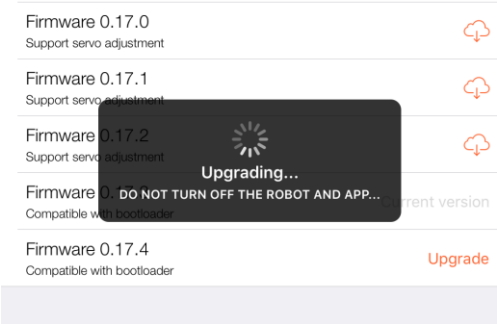

图 6-9 发起固件升级请求

（2）关闭六足机器人的电源

关闭六足机器人电源，主动中断固件下载流程，此时的 program 区域为脏数据，接下来尝试开启六足机器人电源。

六足机器人 LED 蓝色灯长亮，表示已进入了 Bootloader 模式，通过 iOS App 继续升级固件。固件顺利下载完毕。场景 2 测试成功。

3、场景 3：

场景 3 描述的是用户烧写了一个错误的 program 程序进入 program 程序区，而此程序为非法程序，未实现改写 bootflag 的功能，所以，在下次上电开启后，必须进入 Bootloader 程序，并进入待升级模式。

4、场景 4：

场景 4 描述的是一切正常的情况下，在线对六足机器人的固件进行升级。六足机器人上电启动后，必须可以支持在线升级，升级过程应该与场景 1 相同，升级的结果亦需要一样。

6.3 iOS 客户端控制台舵机控制测试

6.3.1 测试方法

获取六足机器人的当前状态，电池图标显示当前的电量信息。另外点击同步按钮，可以获取当前的舵机角度，并实时显示于舵机控制区域。

6.3.2 测试结果

测试结果如图 6-10 至 6-11 所示：

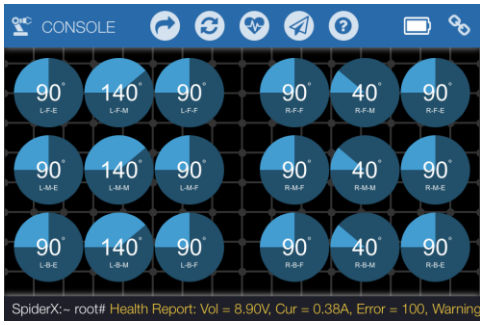

图 6-10 舵机控制界面

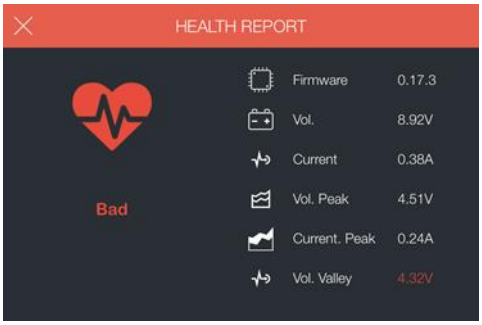

图 6-11 实时健康报告界面

从图 6-10 的舵机角度值可知，app 可以获取当前六足机器人每个舵机的角度数据，同时从图 6-11 获取的电压电流值与实际值相符，测试结果符合要求。

6.4 iOS 客户端 MotionEditor 动作编辑测试

6.4.1 测试方法

动作编辑器需要实现动作文件的新建，删除，编辑和保存操作。可以支持远程查看六足机器人的当前文件列表，并支持删除操作。

6.4.2 测试结果

打开六足机器人的动作添加

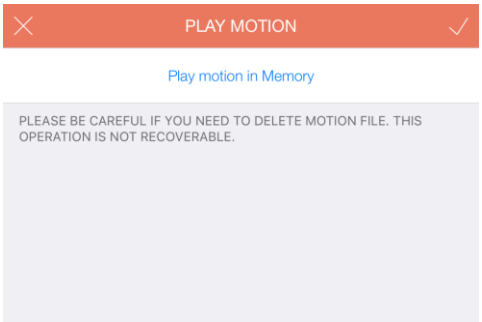

图 6-12 下载站立动作前文件列表

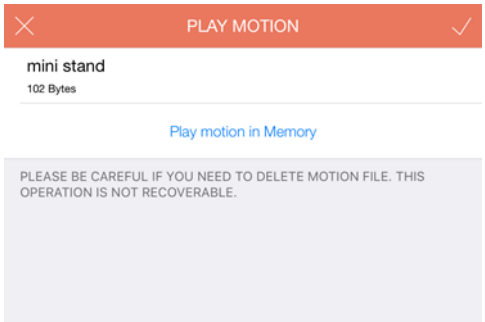

图 6-13 下载站立动作文件后文件列表

下载站立动作前，下位机保存的文件列表为空，结果如图 6-12 所示，下载站立动作文件后，文件列表应该只包含一个文件，结果如图 6-13 所示，测试结果符合要求。

6.5 iOS 客户端远程参数配置测试

6.5.1 测试方法

iOS App 支持设置舵机速度的调整，舵机初始化角度配置，舵机最小角度设置，最大角度设置，微调角度设置。角度设置界面如图 6-14 至 6-17 所示：

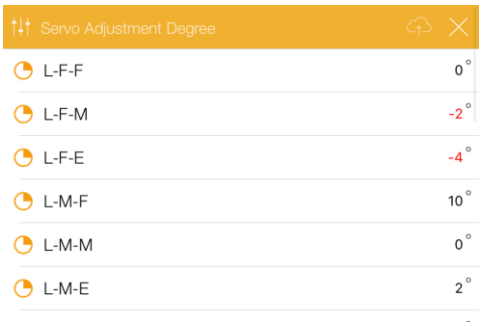

图 6-14 调整角度设置界面

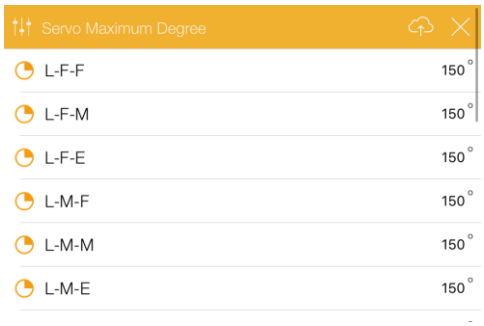

图 6-15 最大角度设置界面

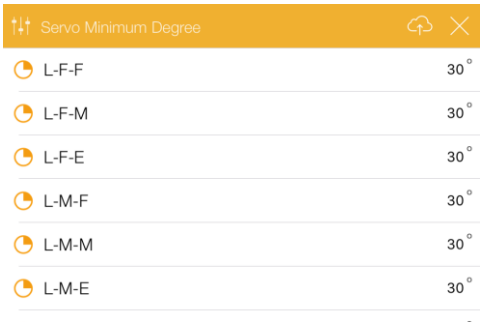

图 6-16 最小角度设置界面

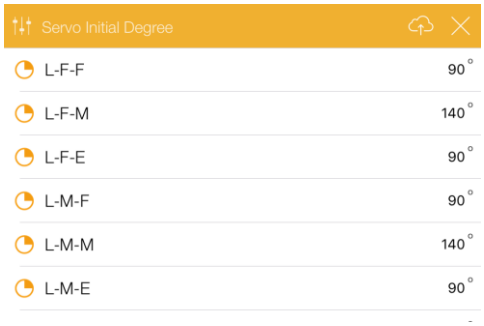

图 6-17 初始角度度设置界面

6.5.2 测试结果

app 设置完成初始角度，最大最小角度和微调角度后，角度值可以保存在下位机的 flash 中，测试结果符合预期要求。

6.6 六足机器人动作测试

6.6.1 测试方法

六足机器人由上电姿态变为站立姿态，需要完成一个变换动作，本测试用例在于测试六足机器人是否可以按照给定站立动作帧，完成由上电姿态到站立姿态的变化。

六足机器人上电姿态如图 6-18 所示：

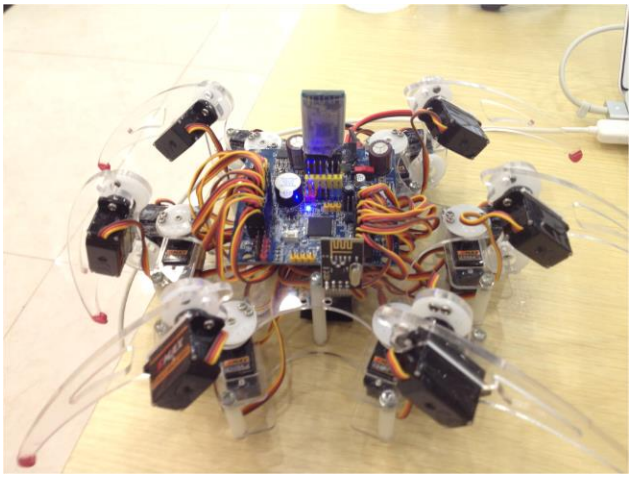

图 6-18 六足机器人初始化完毕姿态示意图

站立姿态如图 6-19 所示：

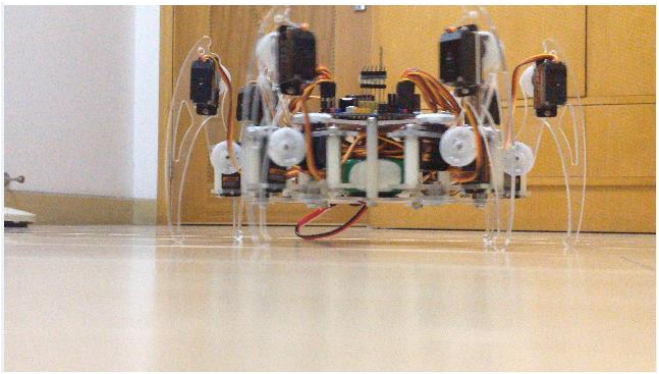

图 6-19 六足机器人站立姿态示意图

站立动作动作帧组合如图 6-20 所示：

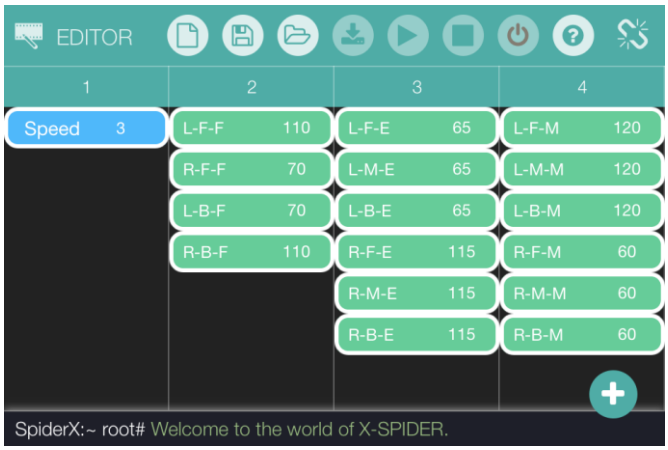

图 6-20 站立动作动作帧组合

六足机器人按照图 6-20 的动作顺利完成站立，测试结果符合预期要求。

6.6.2 测试结果

行走动作基于六足机器人的三足行走法定义动作[19]。每一个前进步骤，移动其中的三只脚，另外三只脚保持站立状态，移动完毕后，再移动另外三只脚，实现交替向前的动作[20-22]。具体动作如图 6-21 所示：

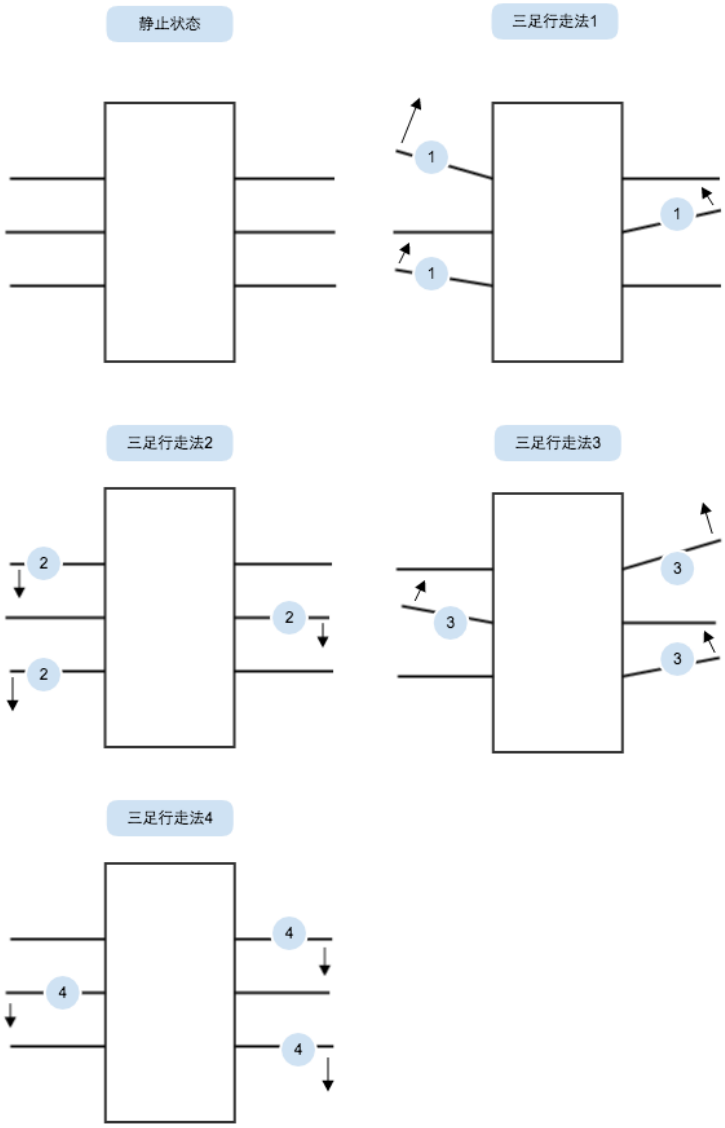

图 6-21 六足机器人行走步态示意图

三足行走法的对应动作文件如图 6-22 至图 6-25 所示：

图 6-22 行走动作帧定义片段 1

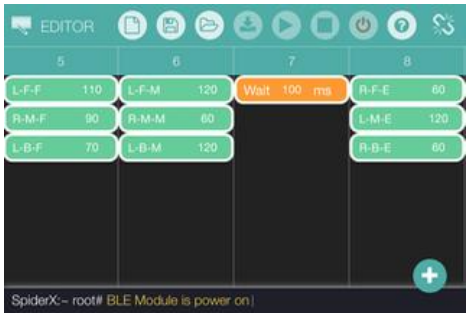

图 6-23 行走动作帧定义片段 2

图 6-24 行走动作帧定义片段 3

图 6-25 行走动作帧定义片段 4

点击 iOS 客户端的执行按钮，实现动作播放，并观察六足机器人是否按三足行走法行走。六足机器人执行行走动作姿态如图 6-26 至 6-29 所示：

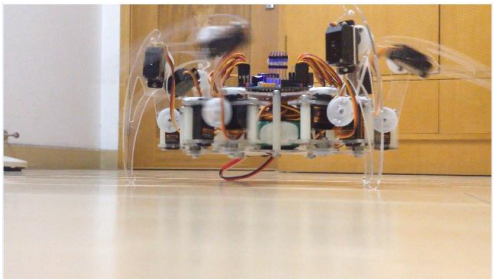

图 6-26 行走动作姿态 1

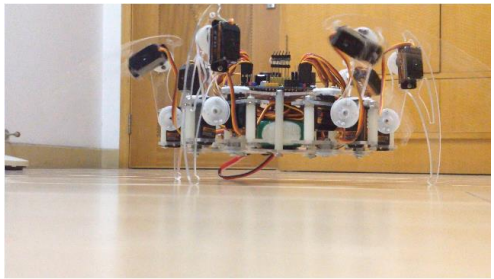

图 6-27 行走动作姿态 2

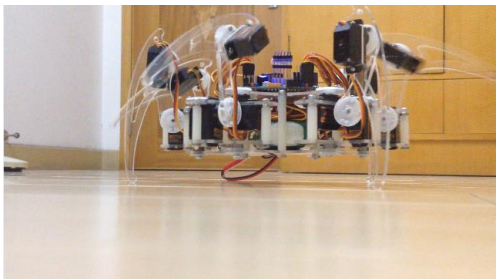

图 6-28 行走动作姿态 3

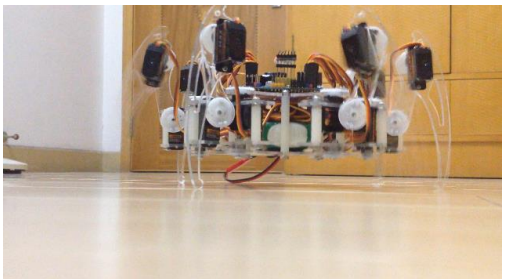

图 6-29 行走动作姿态 4

6.7 本章小结

本章主要阐述 iOS 客户端与六足机器人的集成测试方法与测试结果分析。测试包括 iOS 客户端与六足机器人的蓝牙通信开始测试，在线固件升级测试，远程参数配置测试，动作测试。根据测试结果得知，六足机器人的在线升级功能可以适应各种异常情况，当升级失败后，必须保证进入升级模式，而不再引导固件程序，从而保证下位机在任何时候都可以顺利启动。同时动作帧的自由组合的方式十分的灵活，可以让用户根据自己的实际需求，实现动作的可视化编排。但是测试结果中，仍然存在一些不足，例如 App 的动作编辑器仍然欠缺友好性，操作仍旧略显繁琐，在后续的版本升级中需要改善这个问题。

第 7 章 结论与展望

目前市面上机器人的入门门槛都相对较高，没有一定的知识背景和编程背景的用户，一般很难上手，针对这种状况，本项目通过以 iOS App 为核心，设计并实现了一款支持可视化操作的六足机器人，并尽可能的降低初学者入门的难度，让更多的用户可以接触并了解机器人是如何工作的，并且对没有编程背景的用户，通过使用本项目的 App，可以逐步了解简单的编程思想。

本项目的六足机器人采用 STM32F103 为核心的 MCU，并通过搭配蓝牙 4.0 模块实现与 iOS App 的通信。六足机器人下位机系统采用基于 FreeRTOS 操作系统，实现分层开发，并尽量减少模块间的耦合度。下位机设计了一个 Bootloader 程序，方便用户通过上位机实现在线升级功能。减少用户升级过程带来的麻烦。同时六足机器人实现了一套动作文件协议，用户可自由定义动作帧序列，通过 4 种动作指定的自由搭配，编写丰富的机器人动作。同时六足机器人人下位机实现了一个简易的面向动作文件的文件系统，用户可通过文件系统模块 RBFS，查看当前的文件，删除文件，创建文件操作。为了让用户实时捕获机器人的实时状态，下位机同时实现了电压和电流的采样功能，让用户实时把握六足机器人的最新状态，同时支持软件硬件过压/过流监测，从而降低舵机因堵转而烧毁的概率。

另外，上位机控制端建立与 iOS 操作系统之上，借助 iPhone 强大的图形处理功能和内置的蓝牙模块，实现对六足机器人的下位机的无线控制。上位机让繁琐的舵机操作变为可视化的方式。

最后对六足机器人的下位机与 iOS 上位机执行集成测试，分别对蓝牙命令帧，动作文件执行单元，iOS 上位机各个 UI 模块执行测试，确保项目满足各项需求。

但是项目在各个方面仍有很大的优化空间，例如项目尚未实现摇杆的方向控制，动作编辑器操作仍然比较繁琐，这些问题需要在日后的系统升级与优化中逐步攻克。同时 app 支持的命令仍然很有限，但是对于接下来的研究，可以沿着本文的思路，继续扩展动作指令，让用户可以构建出更加复杂的机器人动作。

参考文献

[1] 张赫. 具有力感知功能的六足机器人及其崎岖地形步行控制研究[D]. 哈尔滨工业大学, 2014.

[2] 魏兰. 移动机器人的可视化编程及仿真系统研究与实现[D]. 武汉理工大学, 2007.

[3] 张梅. 面向机器人应用的可视化开发平台的设计与实现[D]. 哈尔滨工业大学, 2015.

[4] 陈刚. 六足步行机器人位姿控制及步态规划研究[D]. 浙江大学, 2014.

[5] 赵杰，张赫，刘玉斌，等. 六足机器人HITCR-Ⅰ的研制及步行实验[J]. 华南理工大学学报(自然科学版), 2012(12): 17-23.

[6] 范晨灿. 基于蓝牙4.0传输的Android手机心电监护系统[D]. 浙江大学, 2013.

[7] 谢鹏程. 基于STM32和FreeRTOS的独立式运动控制器设计与研究[D]. 华南理工大学, 2012.

[8] 徐金苟. 低能耗蓝牙4.0协议原理与实现方法[J]. 微型电脑应用, 2012, 28(10): 16-19.

[9] 陈诚. 具有半球形足端的六足机器人步态生成和能耗优化研究[D]. 浙江大学, 2012.

[10] 邸文华. 基于iPhone的人员行走惯性导航系统的设计与实现[D]. 天津大学, 2012.

[11] 张静. 基于iPhone的增强现实技术的研究与应用[D]. 电子科技大学, 2010.

[12] Dalrymple M, Knaster S. Learn Objective-C on the Mac[M]. Apress, 2012.

[13] Kochan S G. Programming in Objective-C, Six Edition[M]. 2013.

[14] 杨硕飞. 动态几何画板的研究及其在iPhone平台的实现[D]. 电子科技大学, 2011.

[15] Nutting J, Mark D, Lamarche J, et al. Beginning iOS 7 Development: Exploring the iOS SDK[M]. 2014.

[16] 朱燕生. 基于ARM的全地形八足机器人控制系统研究[D]. 南京理工大学, 2013.

[17] 孙聪. 基于低功耗蓝牙的计步数据传输协议的设计与验证[D]. 福建师范大学, 2015.

[18] 王鑫. 基于BLE和iOS平台的健康管理系统设计与实现[D]. 西安电子科技大学, 2014.

[19] 刘德高. 六足机器人运动控制系统设计与实现[D]. 哈尔滨工业大学, 2013.

[20] 赵小川，罗庆生，韩宝玲. 基于Webots仿真软件的仿生六足机器人机构设计与步态规划[J]. 系统仿真学报, 2009(11): 3241-3245.

[21] 王倩. 六足仿生机器人步态规划与控制系统研制[D]. 哈尔滨工业大学, 2007.

[22] Hoover A M, Steltz E, Fearing R S. RoACH: An autonomous 2.4g crawling hexapod robot[M]. Chatila R, Kelly A, Merlet J P, 2008, 26-33.

致谢

春华秋实两个年头，在职研究生生活不仅忙碌而且非常充实，在忙着工作的同时，周末亦要兼顾学习，熬过了这两年多的时间，回想起来，这段经历还真的让我成长了不少。在这段说长不长说短不短的时间里，从学校学习了很多工作中无法学习到的专业知识。在此，我要向这么多位中山大学的授课老师表达我衷心的感谢。

首先我需要感谢我的导师，您严谨的治学态度，孜孜不倦的回答学生的问题，这种精神非常值得我去敬佩和学习。

感谢我们的教务员，你们的辛勤汗水，不辞劳苦地为我们提供后勤服务，使我们可以更好的专心治学。所以在此必须由衷的感谢你们。

感谢中山大学给了我如此美丽的校园环境和良好的教学氛围，在此，我留下了美好的校园记忆。这些回忆值得我日后再慢慢回味。

最后，感谢我的父母，感谢我的亲人，是你们在背后默默的支持我，才让我取得今天的成绩。